

Datové struktury a algoritmy

Přednáška 9, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Administrativa
- Prezentace: mazání v BVS + adresní vyhledávání
 - Poslední termín, kdy lze prezentovat vlastní téma, je 30.11!
- Bonifikovaný kvíz: Algoritmy vyhledávání a BVS
- Vyvážené vyhledávací stromy
 - Výškově, dokončení (AVL, dokončení a nepovinně RB stromy)
 - Váhově
 - „Přístupově “ (Splay)
- SkipList jako seznamová alternativa k BVS
- Adresní vyhledávání a způsoby jeho implementace
 - Hash table (hašovací tabulka, tab. s rozptýlenými položkami)

Administrativa

- Připomínám:
 - Přesun **povinného** semestrálního kvízu na **7.12.**
 - Přednáška 7.12. se nekoná, zájemci mohou v jejím čase konzultovat
 - Po dohodě i on-line
 - **Povinnou** prezentaci semestrálních projektů **14.12.**
 - Předtermín 21.12.
 - Lze absolvovat i bez zápočtu formou cvičné zkoušky, spojené s pozdějším prezenčním zkušebním termínem (viz Excel, od 7.12.)
 - Přednáška 21.12. se nekoná, zájemci mohou v jejím čase konzultovat
 - Po dohodě i on-line
- Ověření funkčnosti:
 - Semestrálního testu
 - Cvičné zkoušky

Červeno-černé stromy (dobrovolné téma pro zájemce)

- **Červeno-černé stromy (Red - Black Trees)**

Binární vyhledávací strom

Ohodnocení uzlu: klíč

hodnota,

ukazatelé: levý syn, pravý syn, otec

barva: **červená (R)**/ **černá (B)**

- **Vlastnosti R-BT**

1. Každý uzel je buď B nebo R
2. Každý list (NIL) je B
3. Jestliže je uzel R, pak jeho synové jsou B
4. Každá cesta z vnitřního uzlu do následných listů má stejný počet uzlů B

- RB-T s n vnitřními uzly má výšku nejvýše **$2 \cdot (\log(n+1))$**

$\Rightarrow Q(n) = I(n) = D(n) = O(n)$

RB stromy, shrnutí

- Vlastnosti

Uzly R, B, dočasně pro mazání i double B

Root = B

Nový uzel = R

NULLs = B

Všechny větve obsahují stejný počet uzlů B

Neexistují dva sousedící uzly R

- Insert:

B strýc/teta = Rotace a oprava barvy (výsledek: B+R,R)

R strýc/teta = změna barev (výsledek: R+B,B)

- Mazání:

R uzly bez problémů

B uzly s pomocí konverze mezi 6 standardními konfiguracemi

RB strom, barvy listů

Černý list



Červený list



Dvojitý černý list

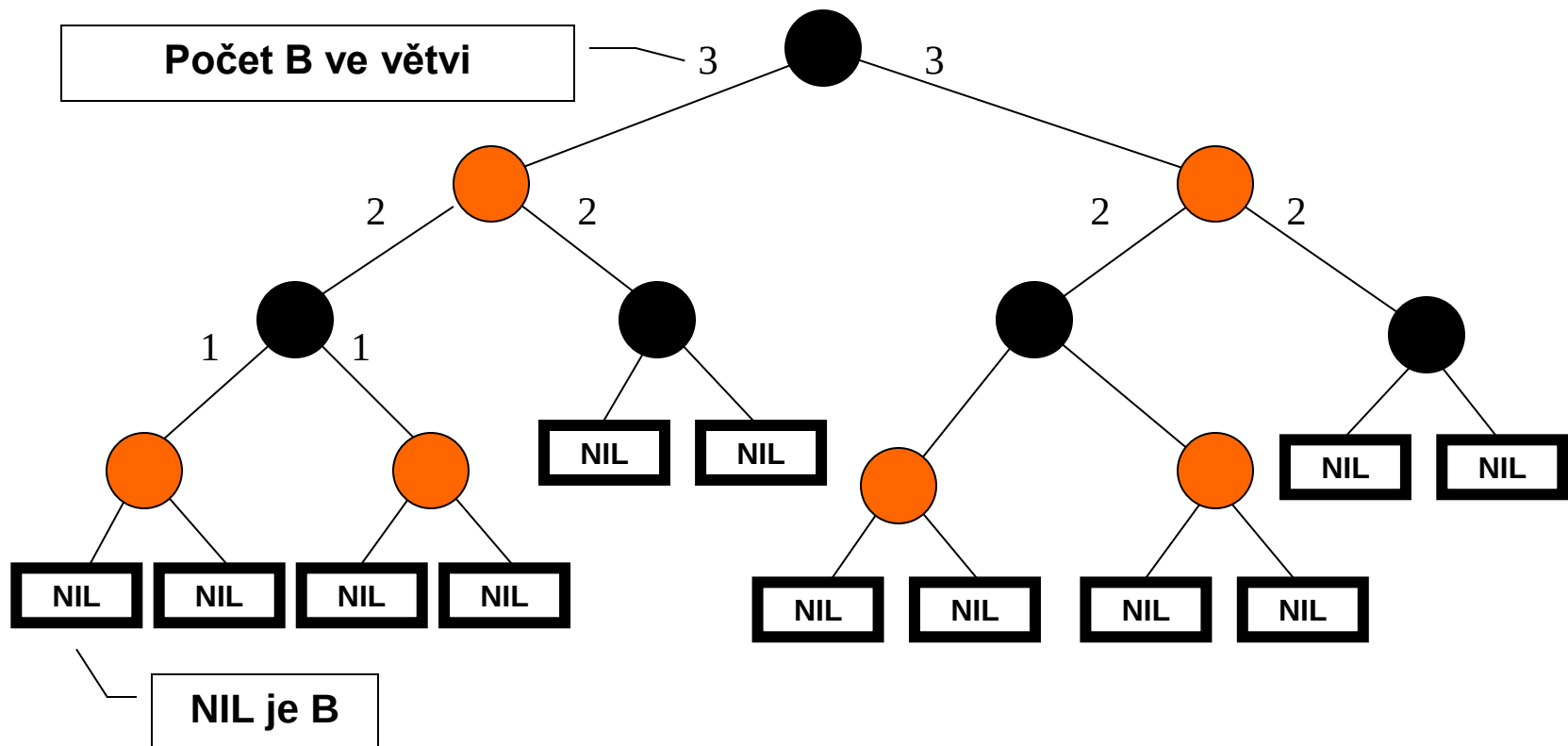


- Zavádí se kvůli prvotnímu vyrovnání černé výšky stromu při mazání černého uzlu s alespoň jedním černým potomkem. Cílem následujících úprav je přebarvení dvojitě černého uzlu na normální černý uzel.
- V některých úpravách nezáleží na výsledné barvě listu, tyto listy jsou označeny modře:



Červeno-černé stromy

- Příklad R-BT



Červeno-černé stromy

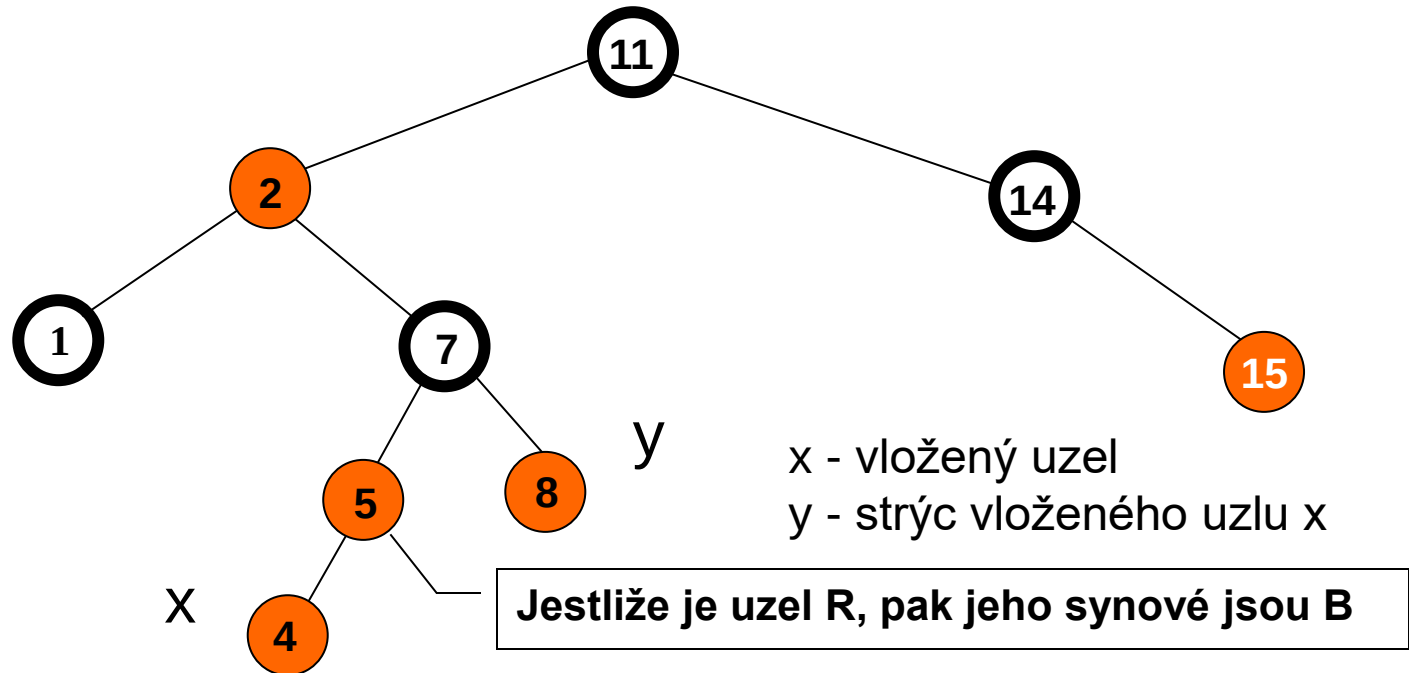
- Operace NAJDI - viz. vyhledávání v BVS
- Operace VLOŽ(T,x)
 1. Vlož x do S - viz. BVS
 2. Přiřaď x barvu R
 3. S použitím operací rotace přebarvi T tak, aby vyhovoval podmínkám 1 až 4 pro R-BT
 $\Rightarrow$ R-BT_VLOŽ (T, x)

Poznámky:

1. Dvojitá rotace je rozložena do dvou jednoduchých.
2. Existuje levá a pravá varianta rotace.

Červeno-černé stromy

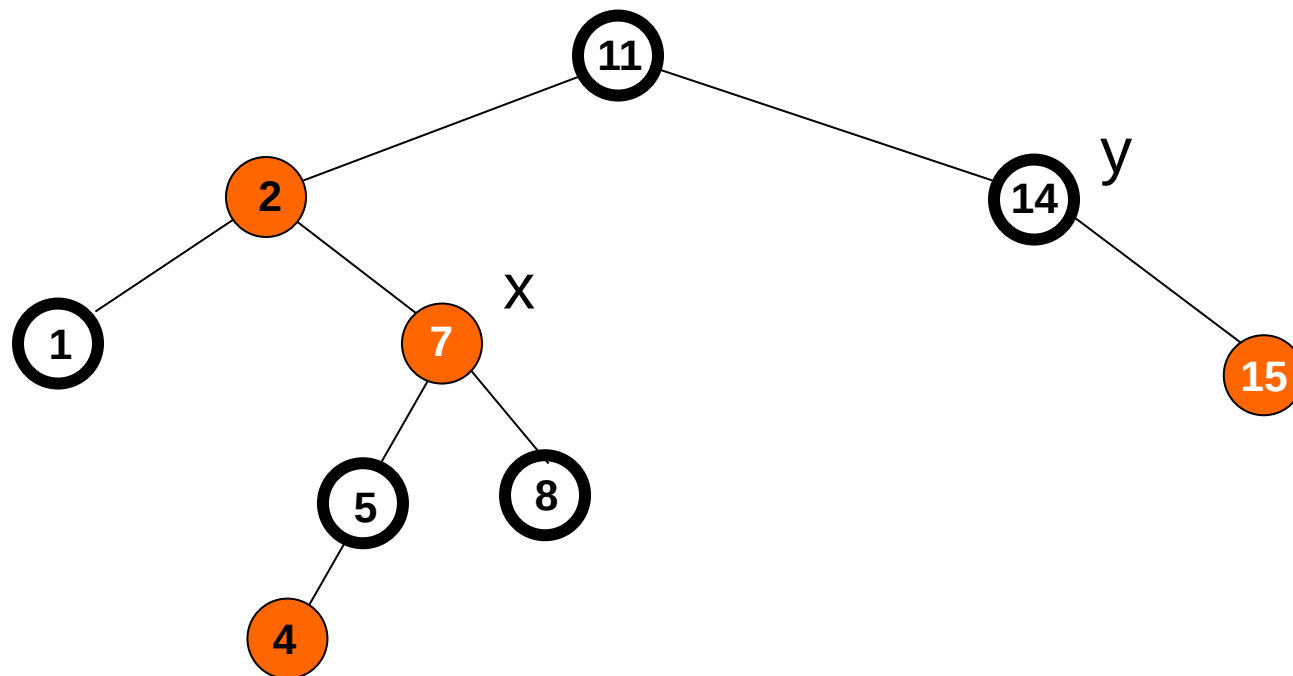
- R-BT po vložení uzlu x (deklarativně R)



- Uzel 5 nesplňuje podmínku 3 $\Rightarrow$ přebarvit 5, 8, a 7
(varianta 1 v algoritmu)

Červeno-černé stromy

- R-BT po přebarvení (podle varianty 1)

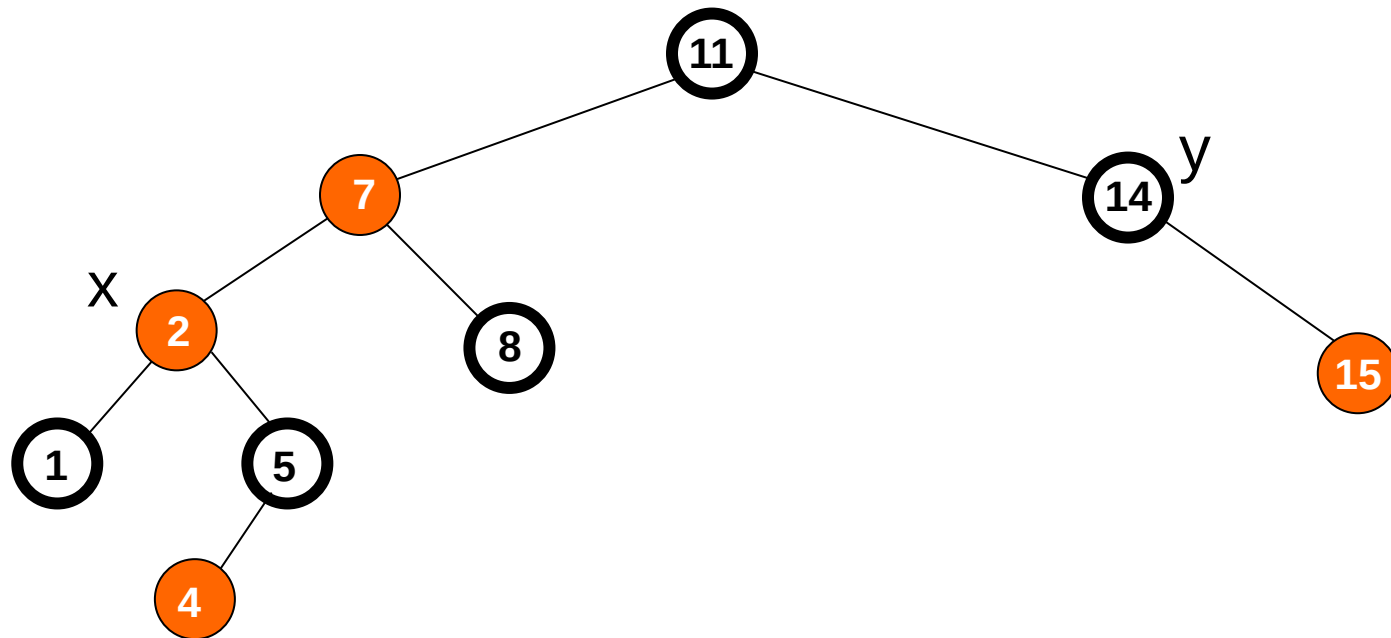


x a jeho otec jsou R
strýc x je B
x je pravým synem

⇒ levá rotace (varianta 2)

Červeno-černé stromy

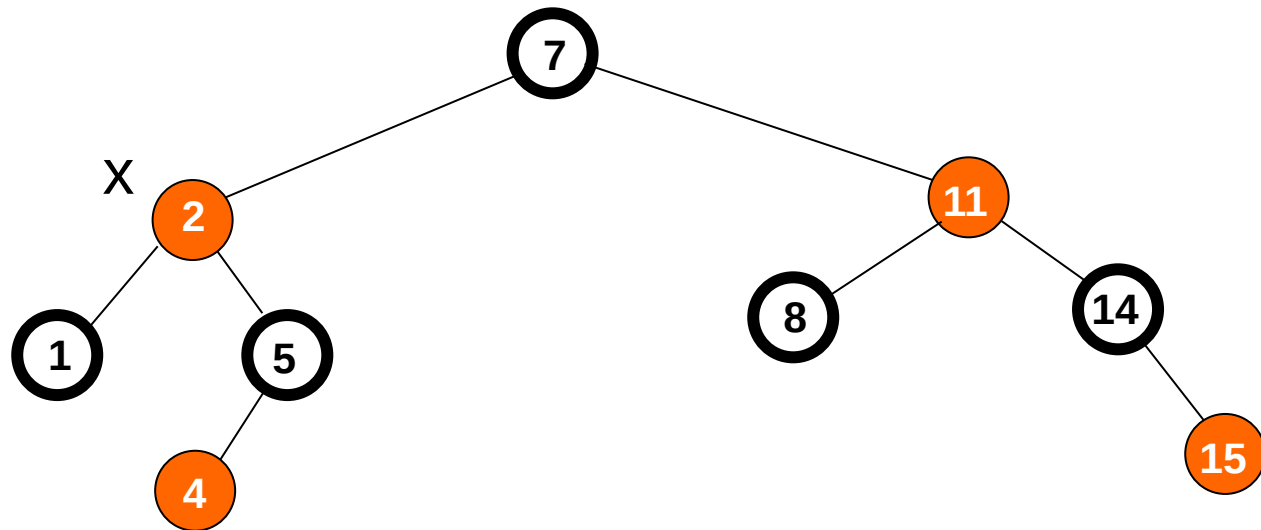
- R-BT po levé rotaci (podle varianty 2)



uzel 7 nemůže být červený $\Rightarrow$ pravá rotace

Červeno-černé stromy

- R-BT po pravé rotaci (podle varianty 3)



Červeno-černé stromy

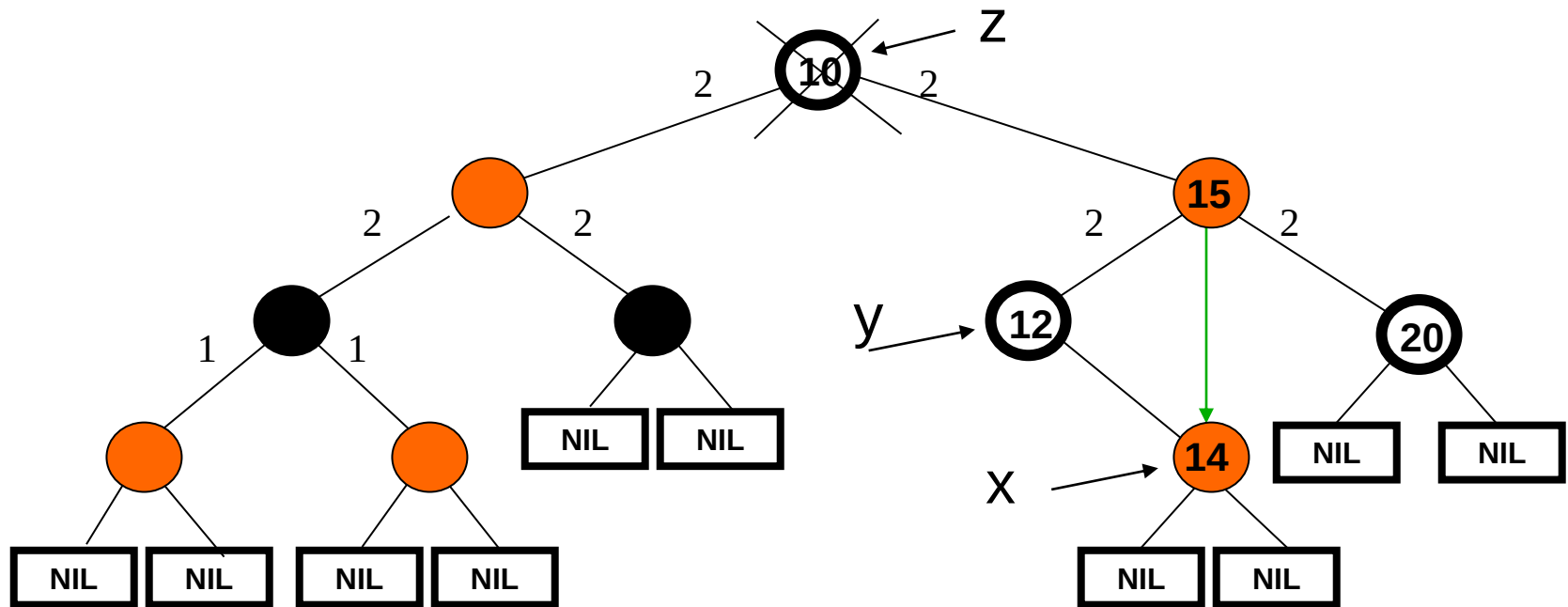
- R-BT_VLOŽ(T, x)
x přiřad' R
while x <> kořen **and** barva otce = R
do if otec je levým synem děda
 then y := strýce
 if barva strýce je R
 then otec := B (varianta 1)
 strýc := B
 děd := R
 else if x je pravý syn
 then x := otce (varianta 2)
 PRAVÁ_ROTACE(T, x)
 otec := B (varianta 3)
 děd := R
 PRAVÁ_ROTACE(T, děd)
 else stejně jako **then** se záměnou stran levý, pravý
 kořen := B

Červeno-černé stromy

- **Rušení prvku v R-BT**
obdoba rušení prvku v BVS
 1. Nalezení rušeného prvku;
 2. **if** rušený prvek z je list
then odkaz u otce uprav na nil
else if rušený prvek z nemá levého syna
then z nahrad' jeho pravým synem
else if rušený prvek z nemá pravého syna
then z nahrad' jeho pravým synem
else z nahrad' nejbližším nižším, resp.
nejbližším vyšším prvkem;
 3. **if** nahrazující prvek je B
then Oprava vybarvení;
- Oprava vybarvení není nutná, když nahrazující prvek je R
(nenaruší pravidlo B-výšky, žádné dva R nejsou sousední)

Červeno-černé stromy

- Situace před náhradou nejbližším vyšším



z - rušený prvek
y - nahrazující prvek
x - syn nahrazujícího

Červeno-černé stromy

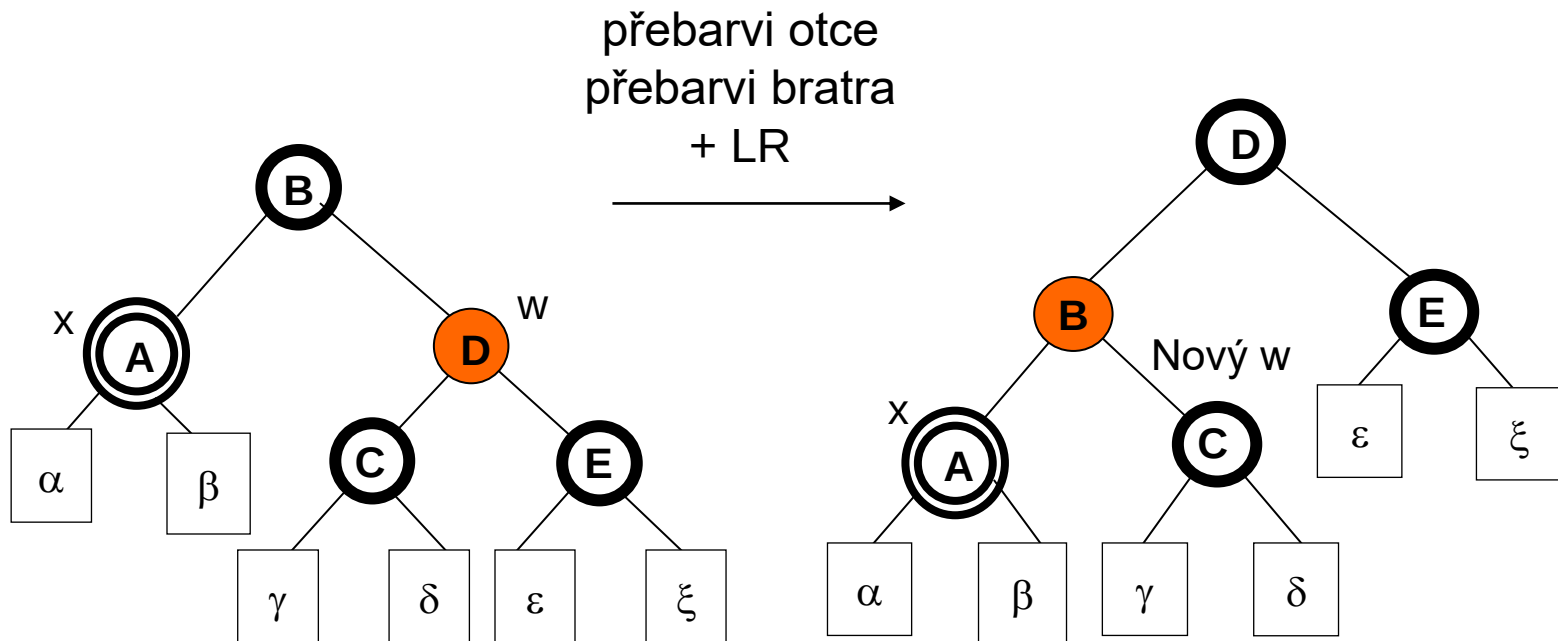
- **Varianty úpravy vybarvení**

Poznámky:

1. Přebarvení se provádí operací rotace
2. zelené uzly - uzly, které mohou být obarveny R/B
označení c nebo c'
3. $\alpha, \beta, \gamma, \delta, \varepsilon, \xi$ - libovolné podstromy
4. x - syn nahrazujícího
extra B počet černých uzlů cesty přes x se zvyšuje o 1
 w - bratr x

Červeno-černé stromy

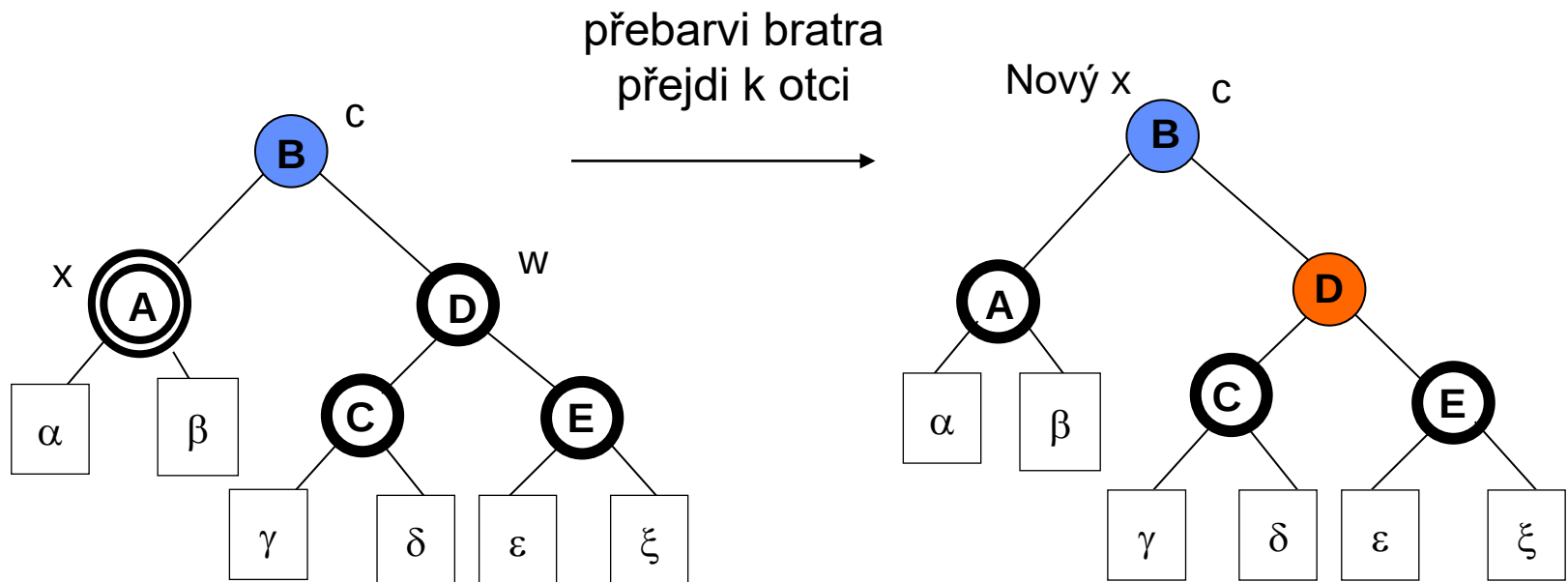
- **Varianta 1:** bratr w (uzel D) je R



Varianta 1 se transformuje na variantu 2, 3, nebo 4

Červeno-černé stromy

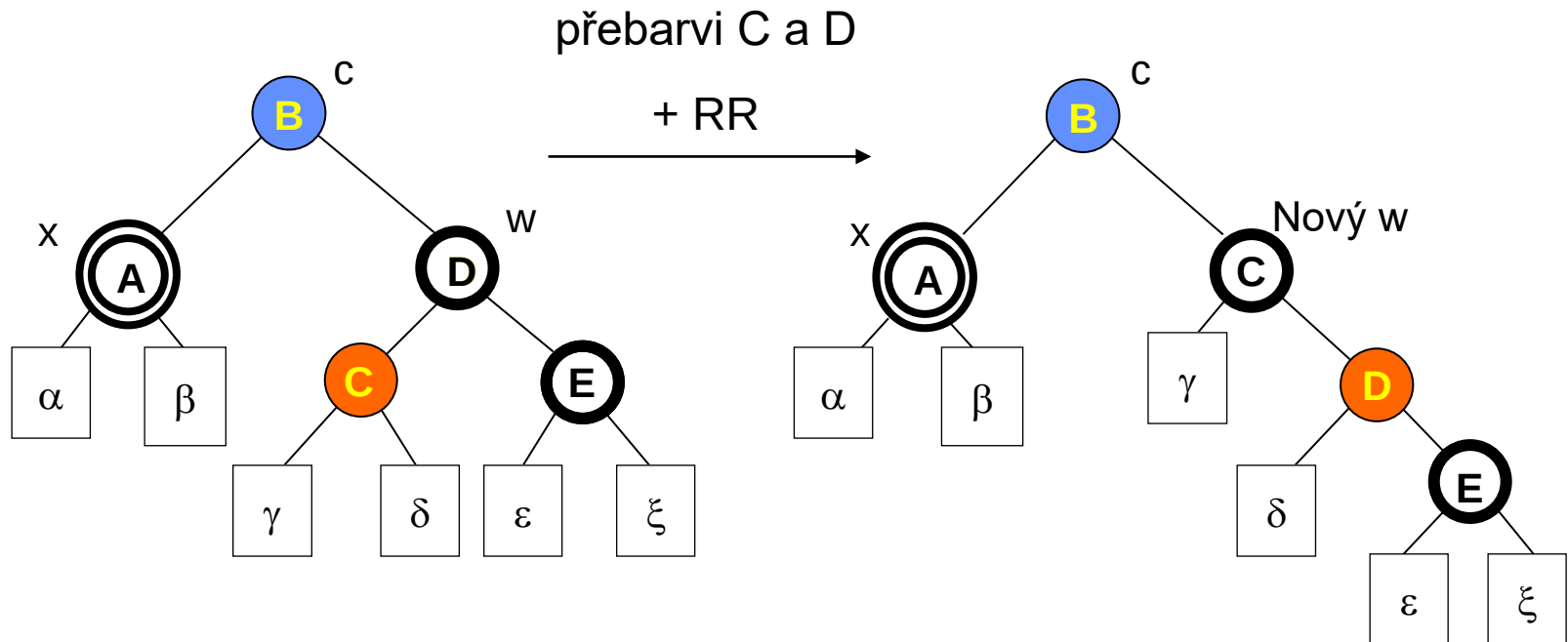
- Varianta 2:** oba synovci (uzly C a E) jsou B



- Varianta 2 se může cyklicky opakovat

Červeno-černé stromy

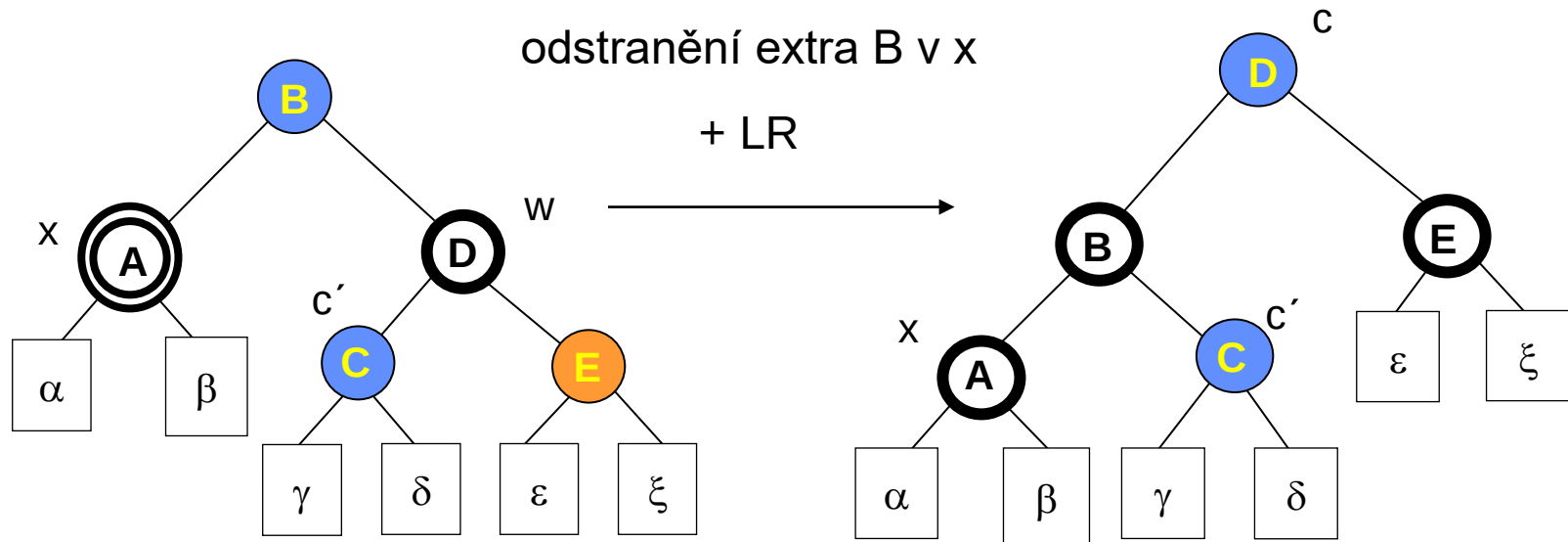
- **Varianta 3:** pravý synovec (uzel E) je B



- Transformace pro variantu 4

Červeno-černé stromy

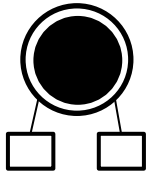
- **Varianta 4: dokončení úprav po variantách 1, 2, 3**



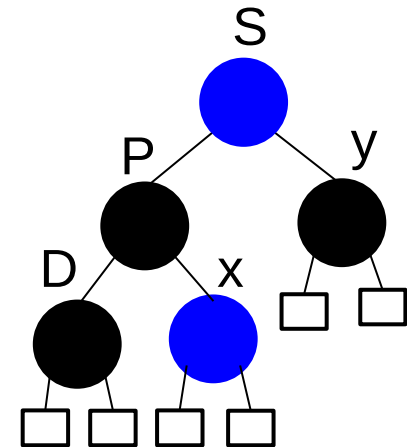
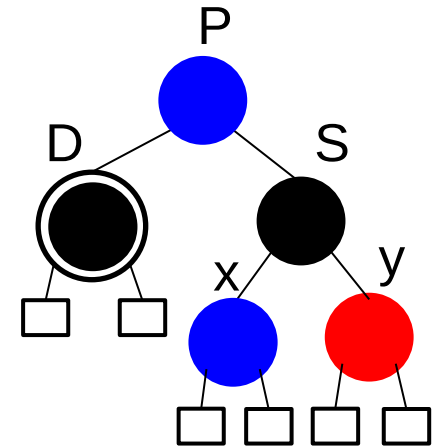
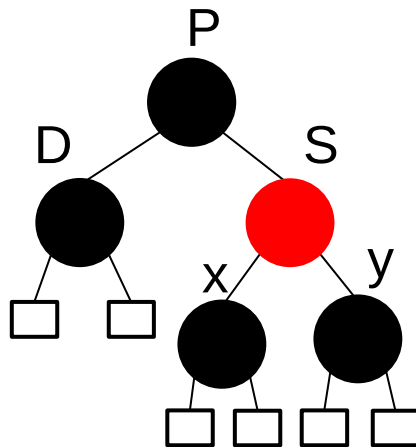
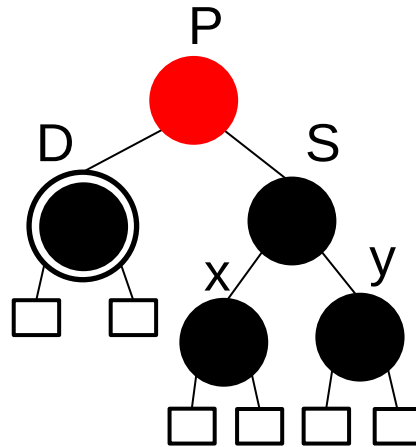
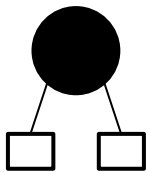
- Nový $x = \text{root}(T)$
- Nakonec se x přiřadí B

Mazání v RB stromu, koncové situace (levá varianta)

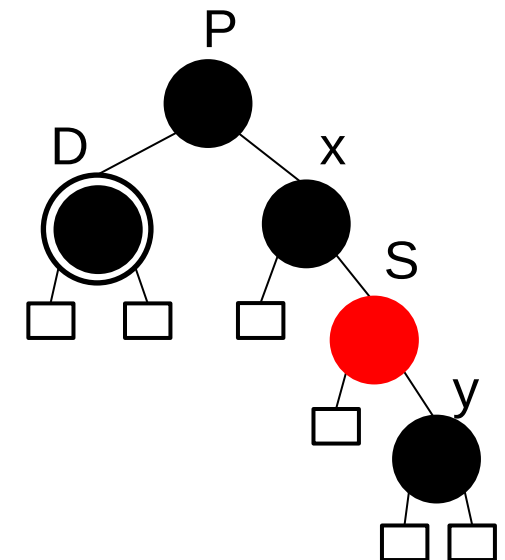
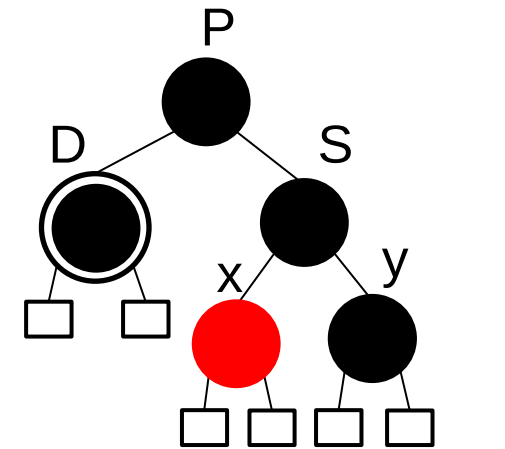
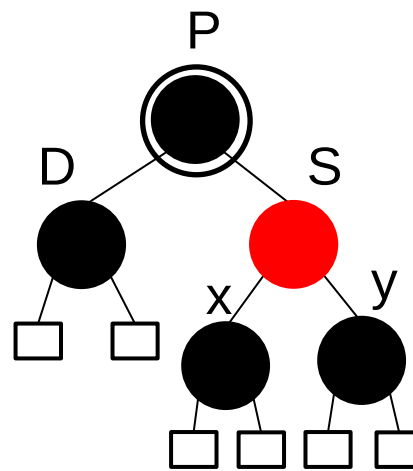
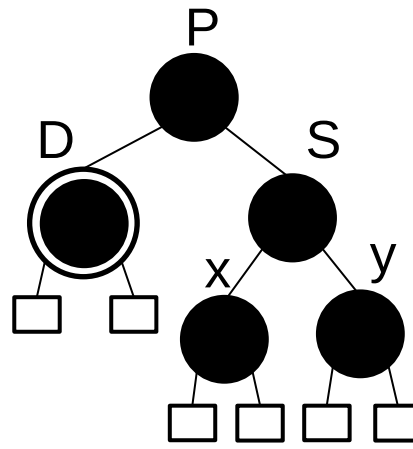
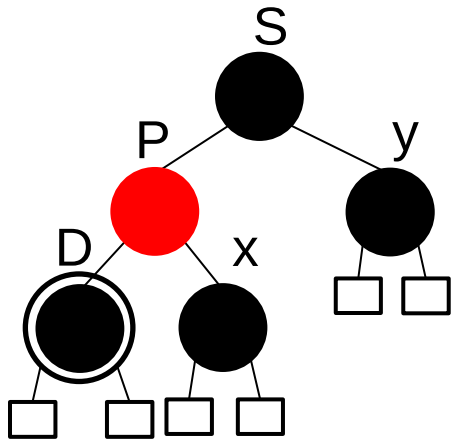
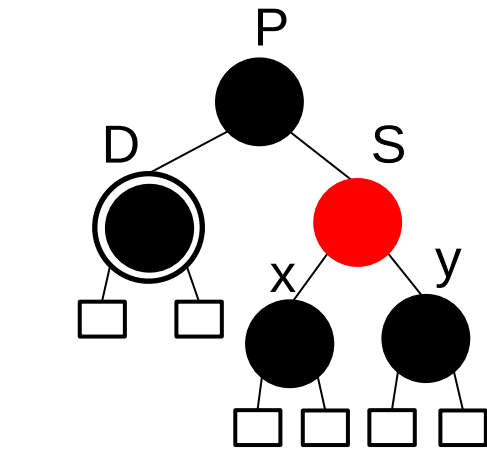
D (root)



D (root)

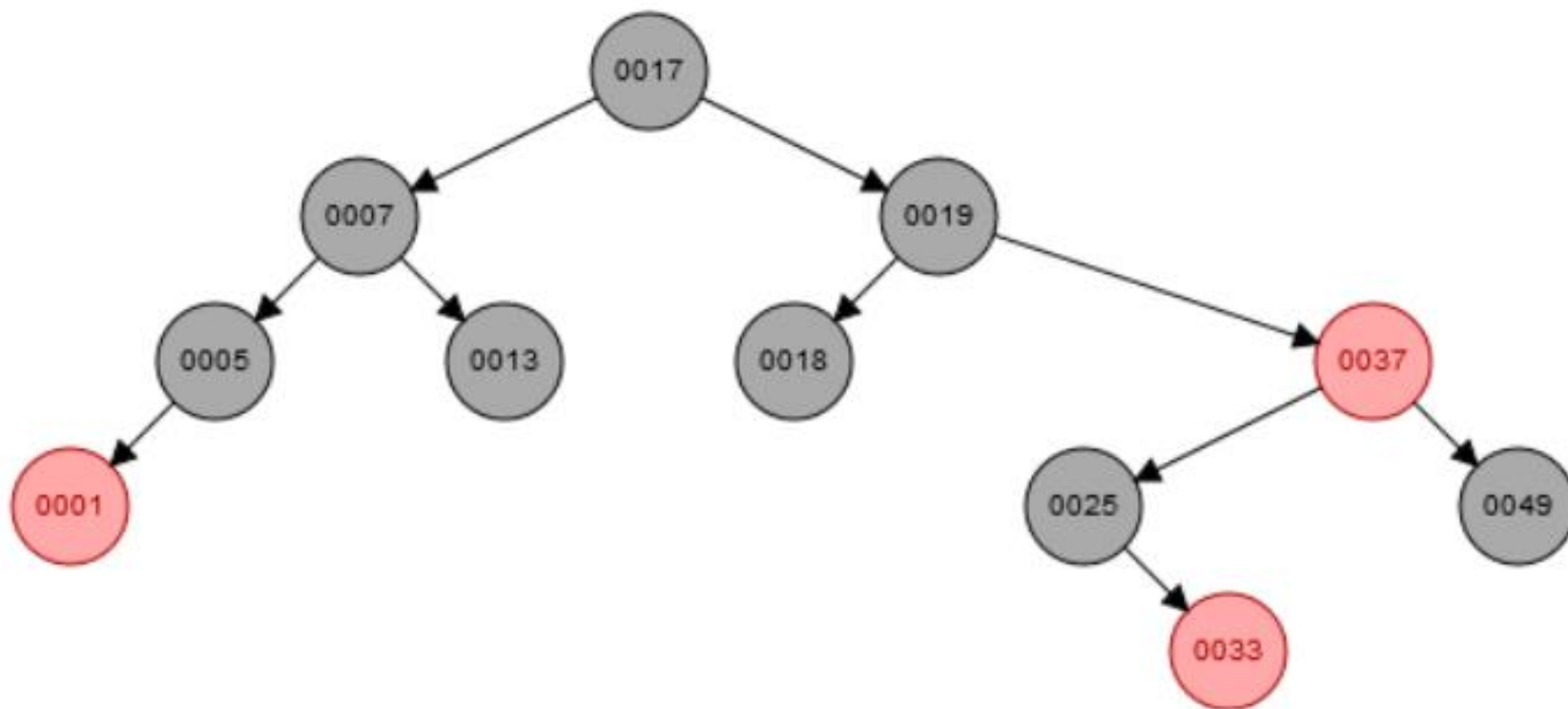


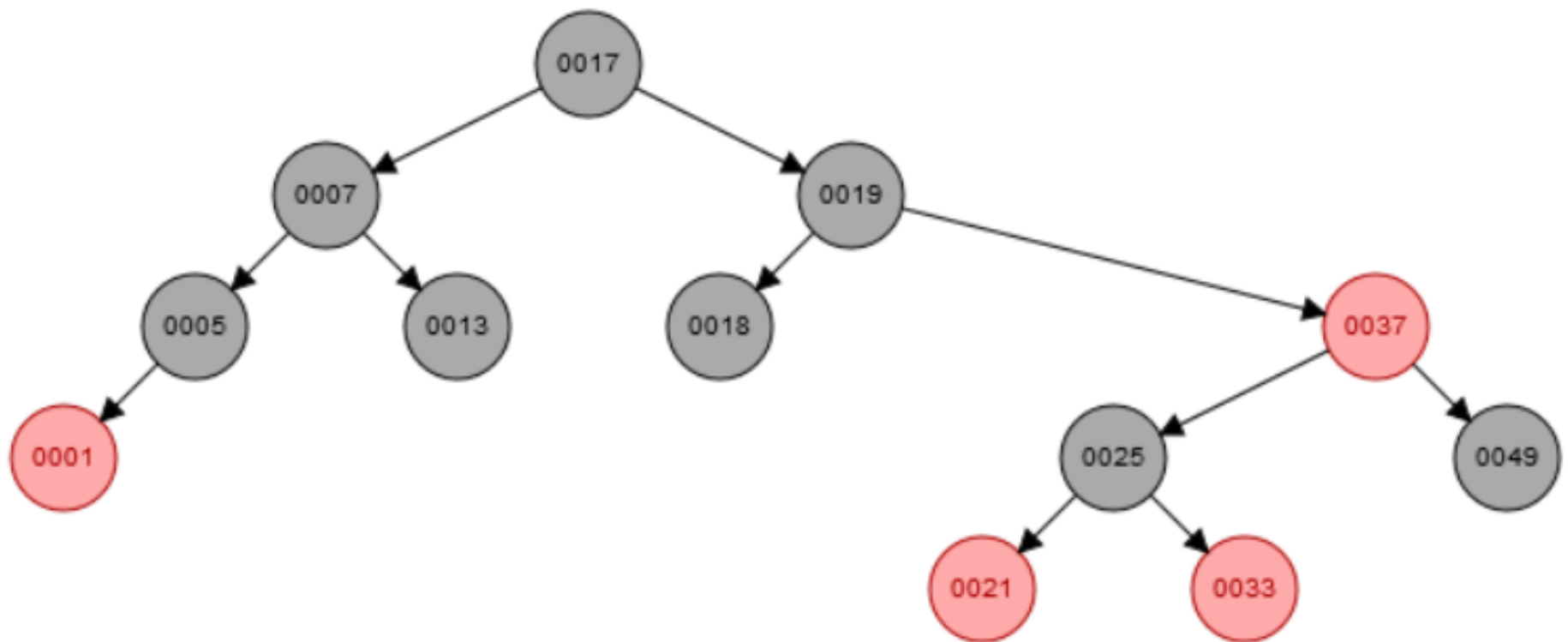
Mazání v RB stromu, průběžné situace (levá varianta)

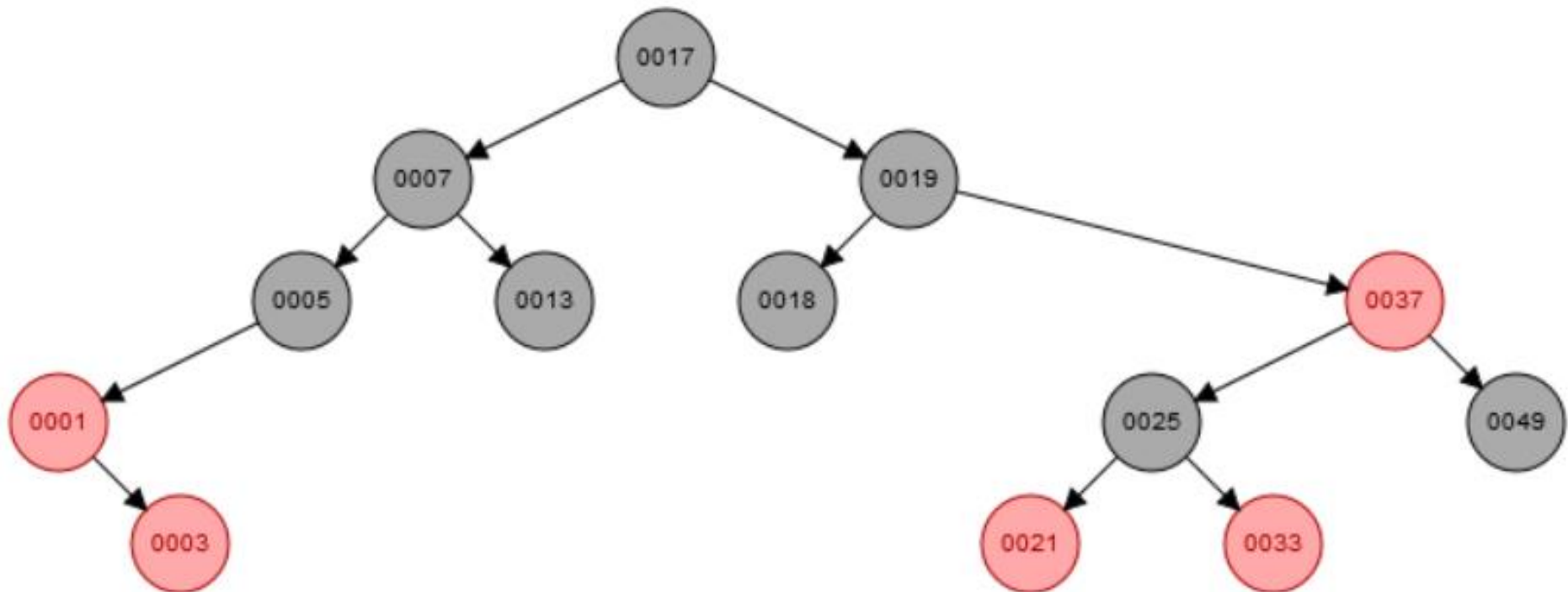


RB strom, příklad 1

1/9

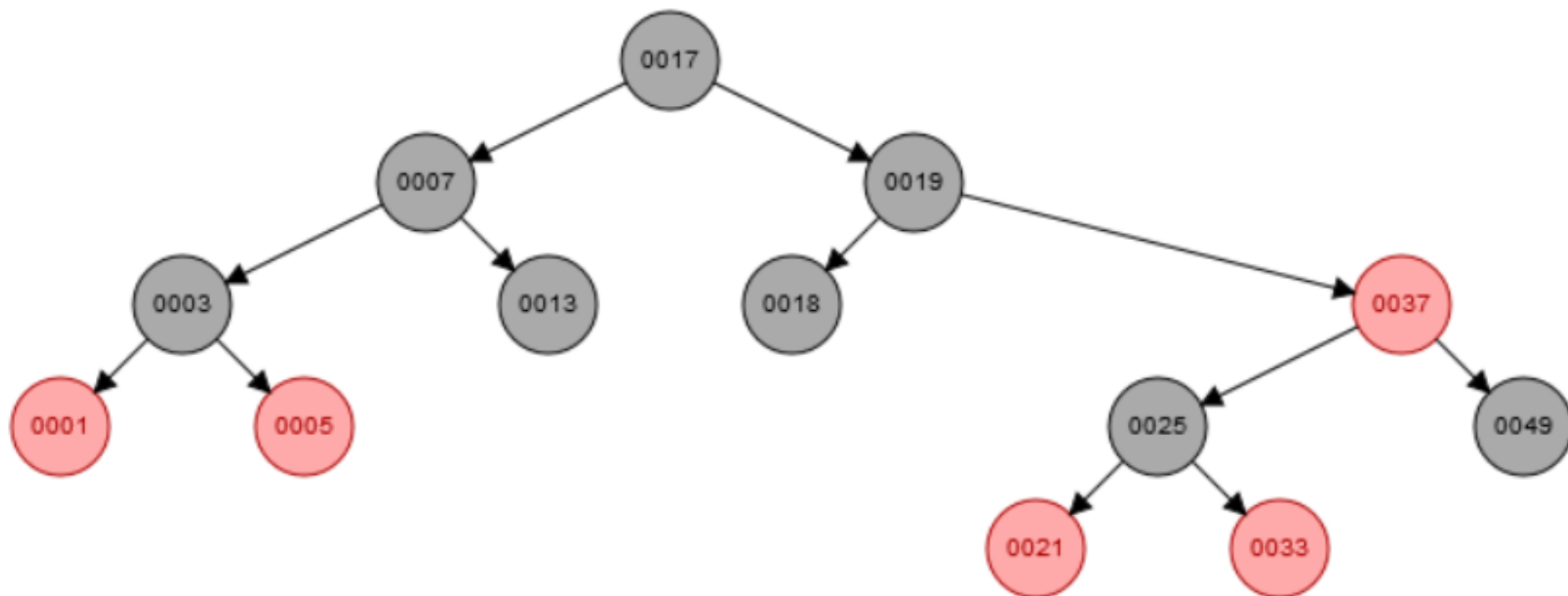






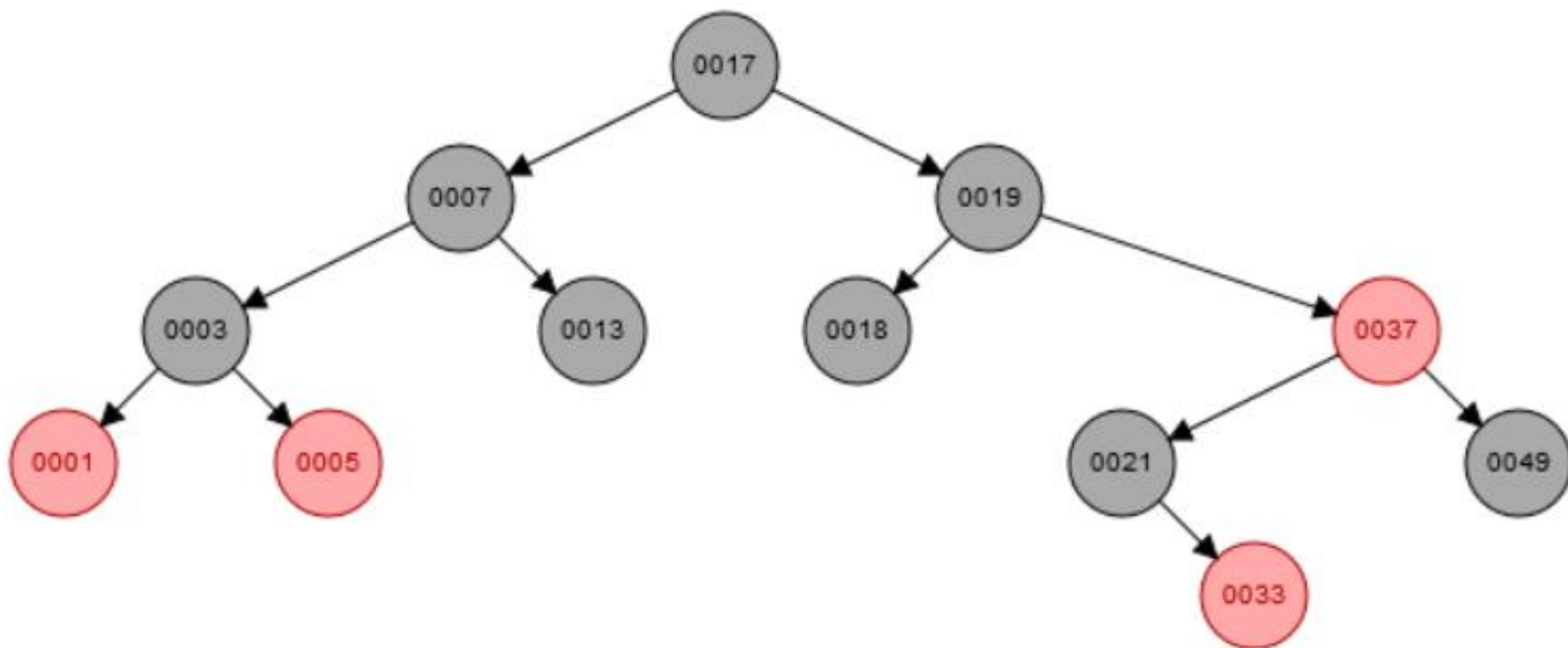
... přebarvi a vyvaž.

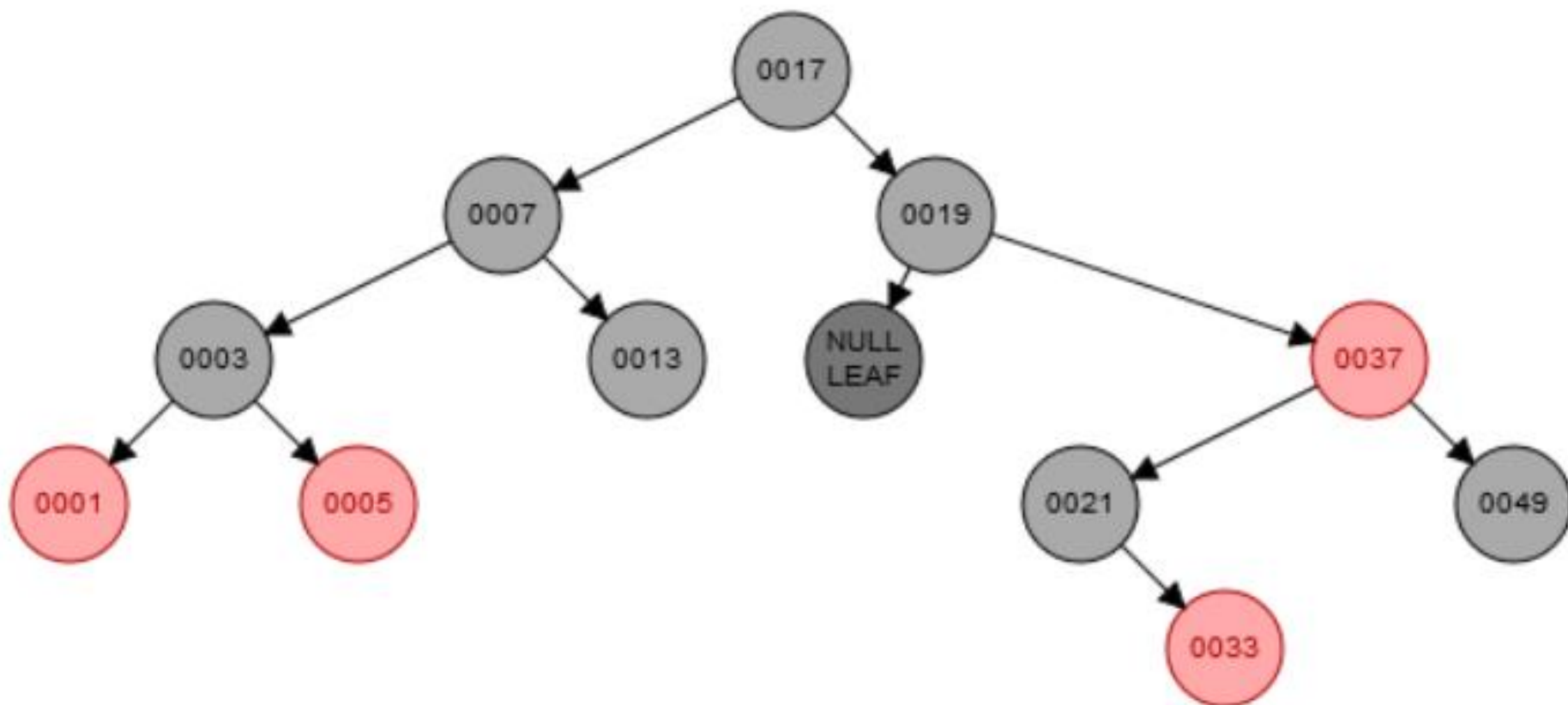
4/9

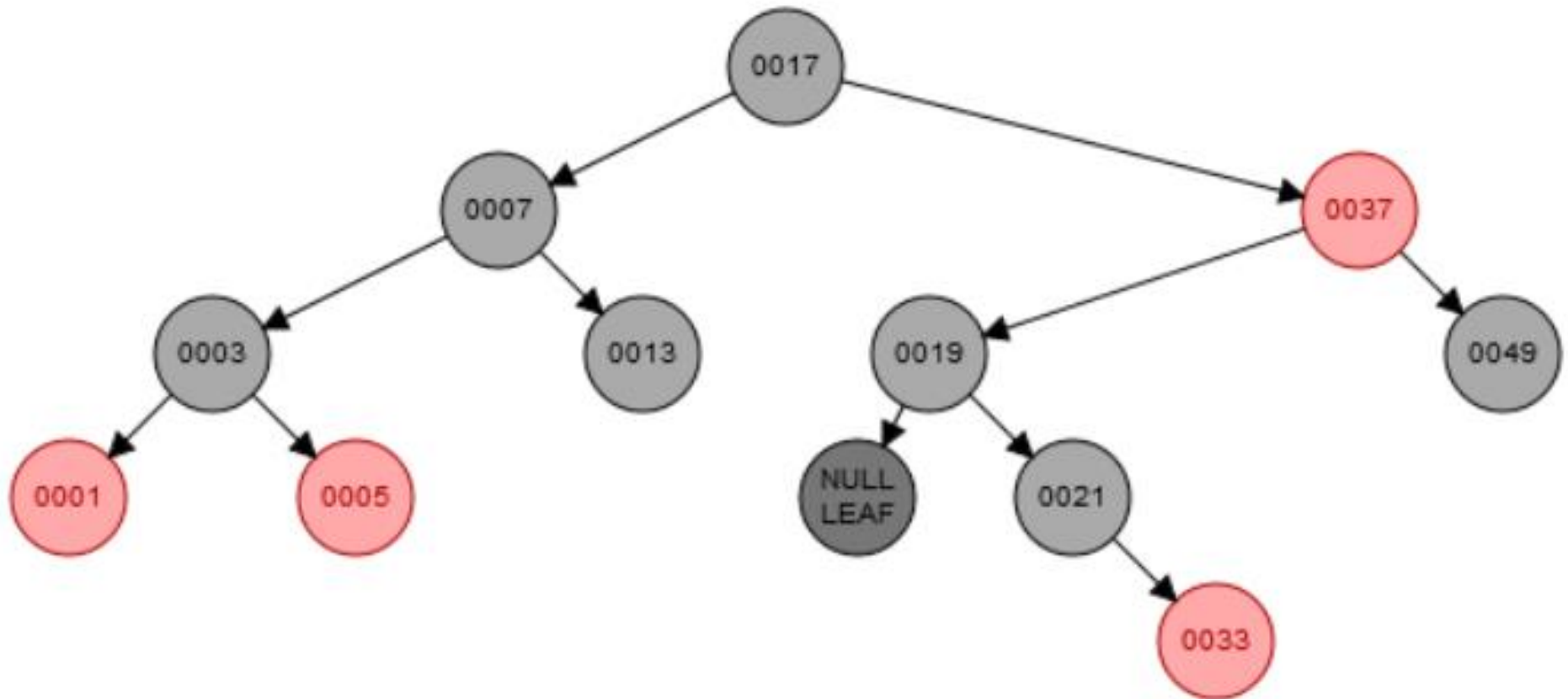


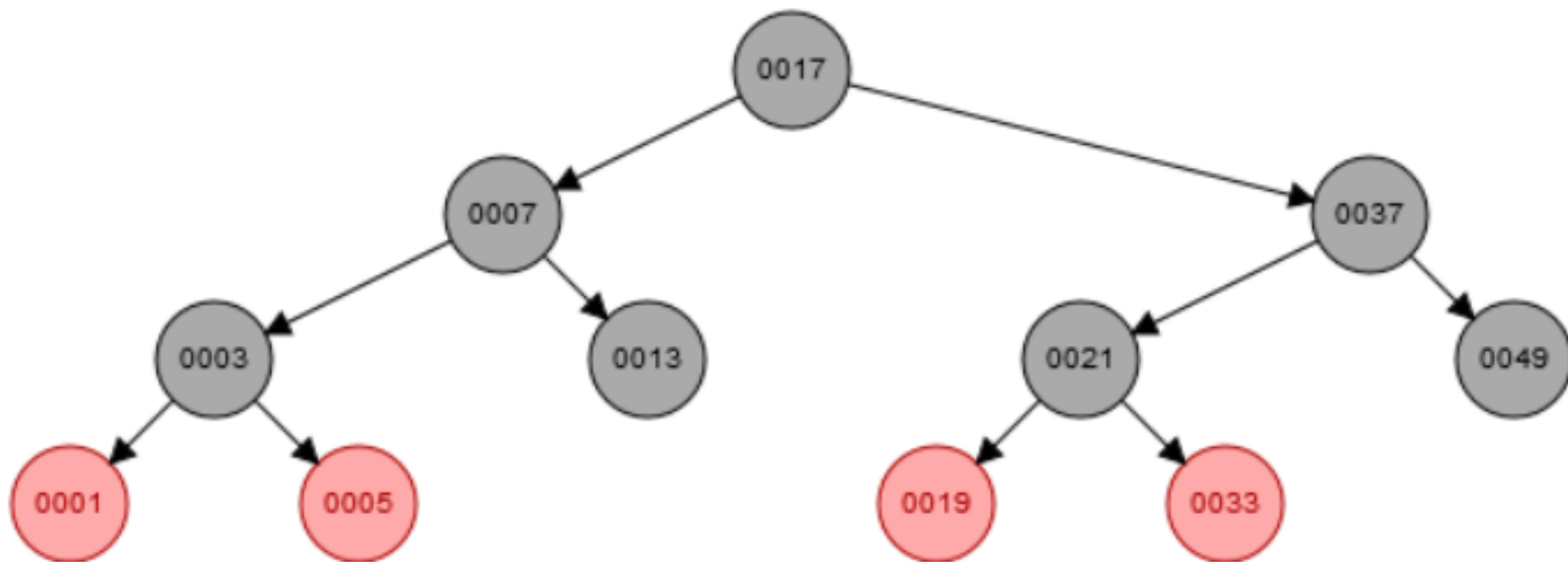
Odstraň 25 a přebarvi

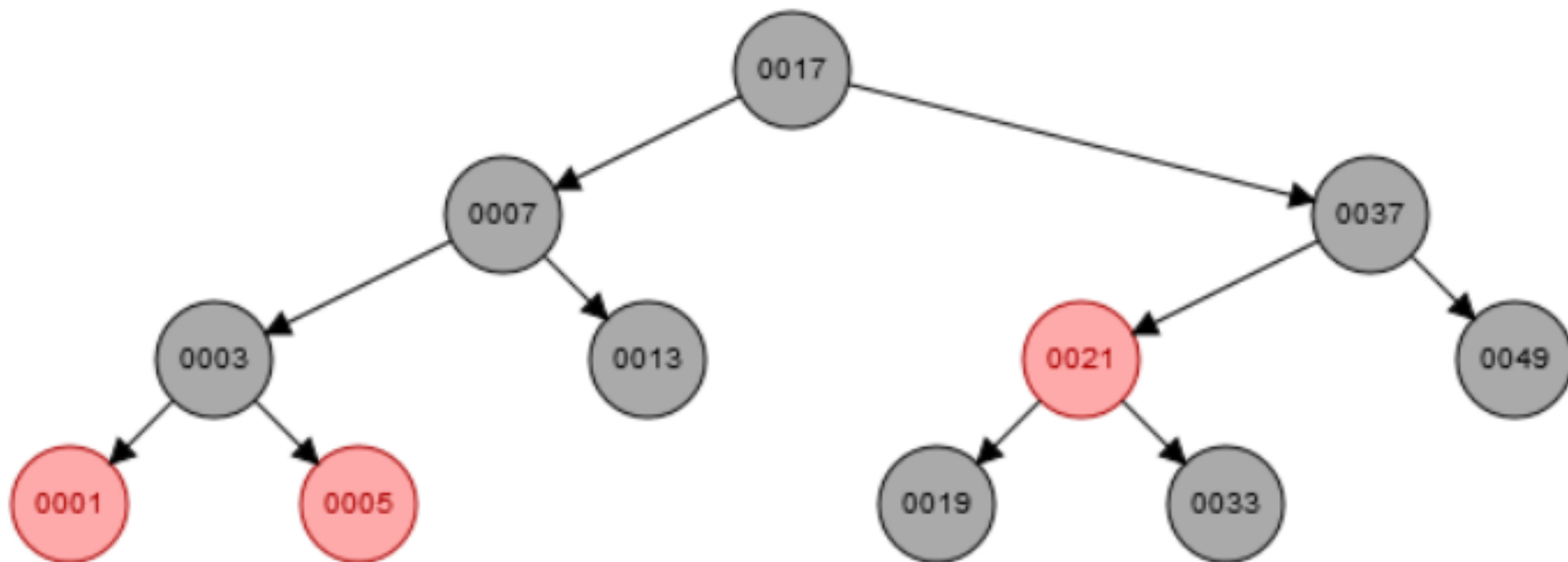
5/9





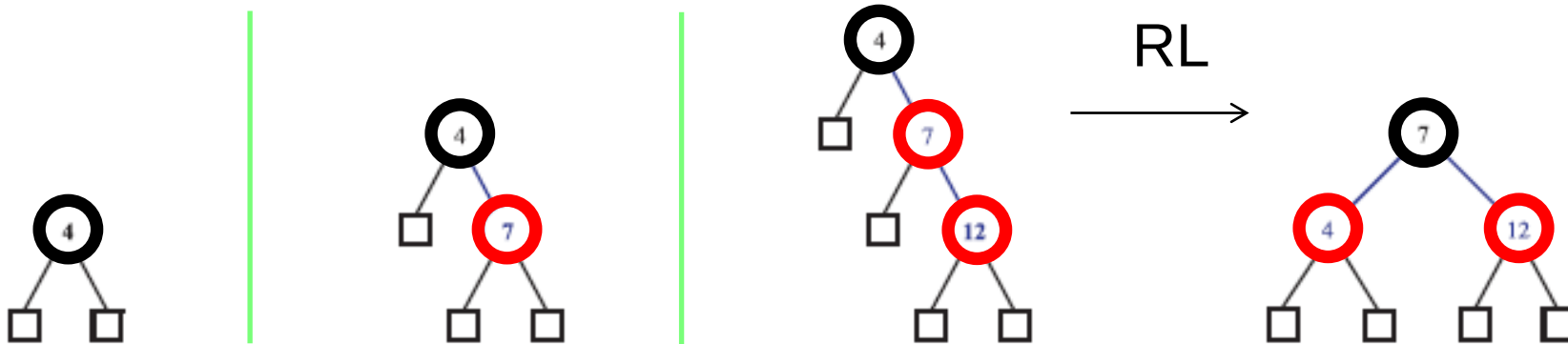






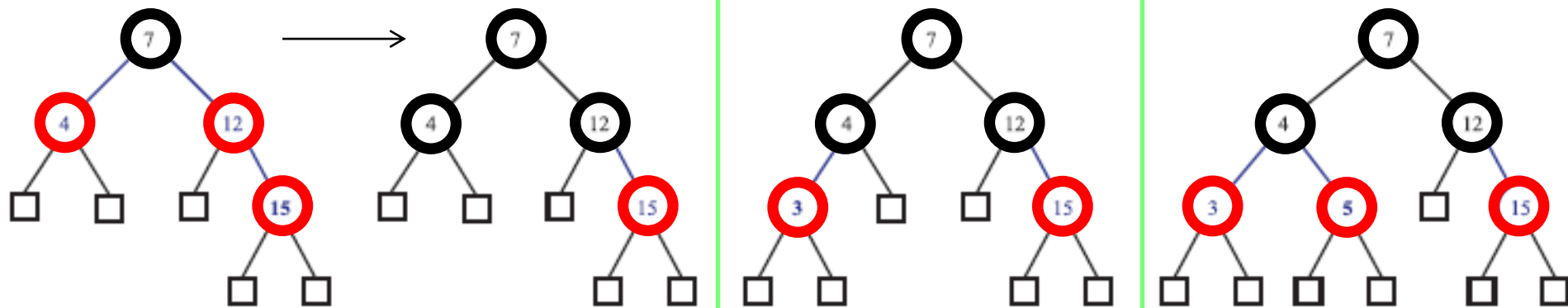
RB strom, příklad 2 : vlož 4,7,12,15,3,5

1/5

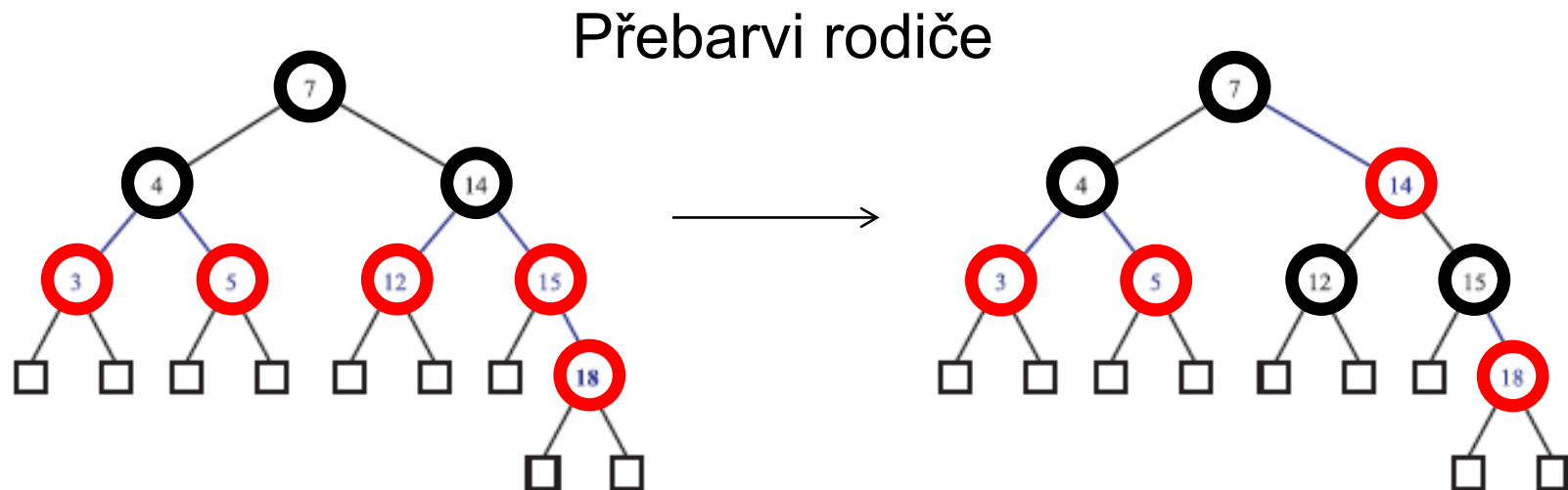
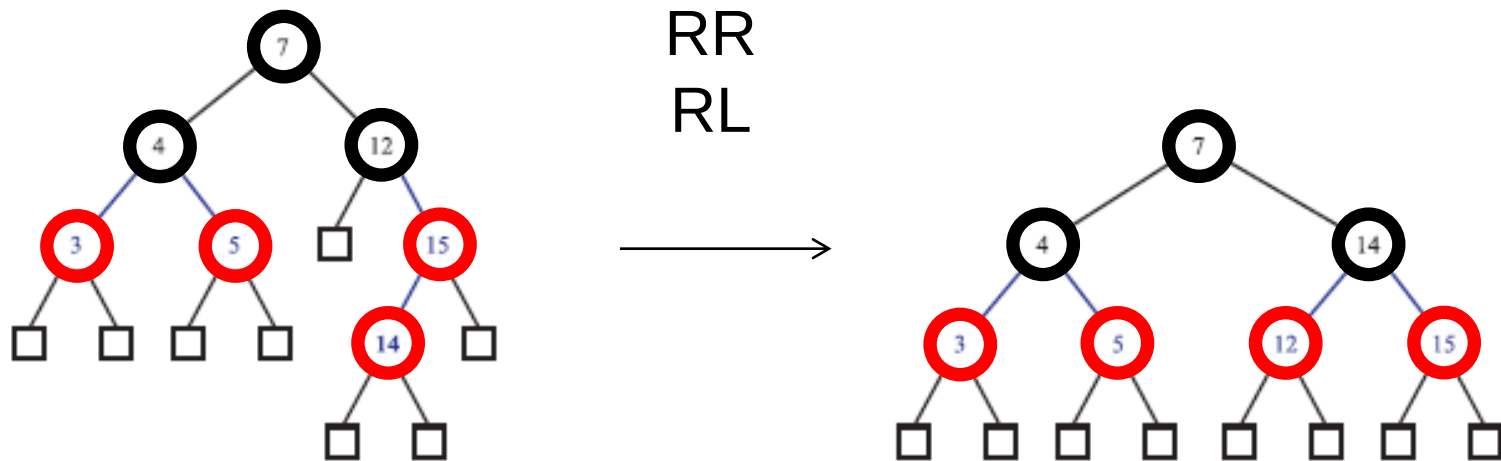


Přebarvi rodiče

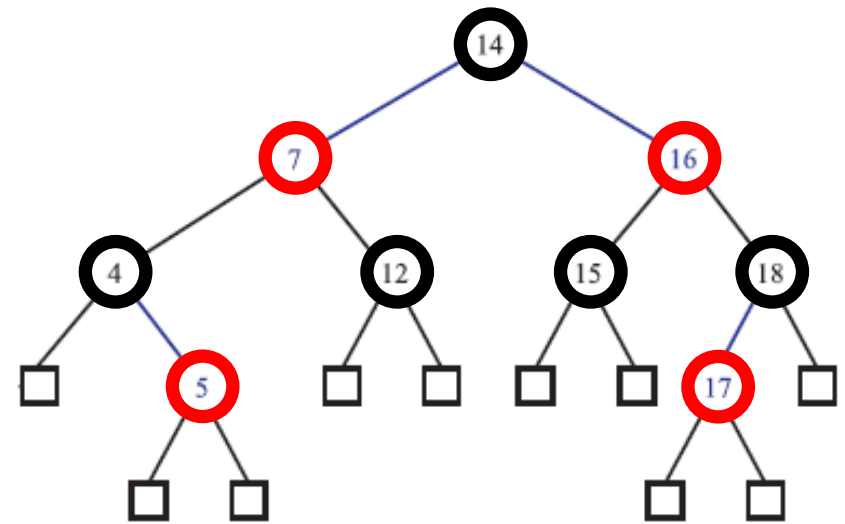
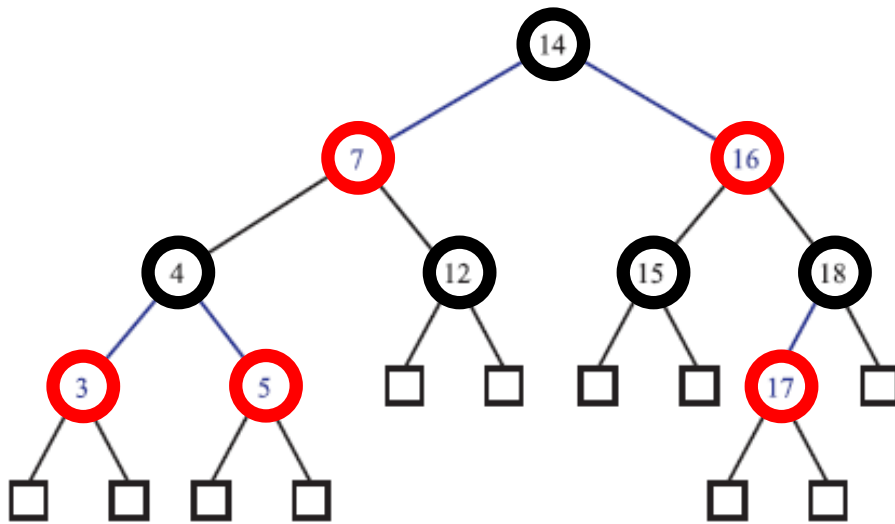
Přebarvi kořen



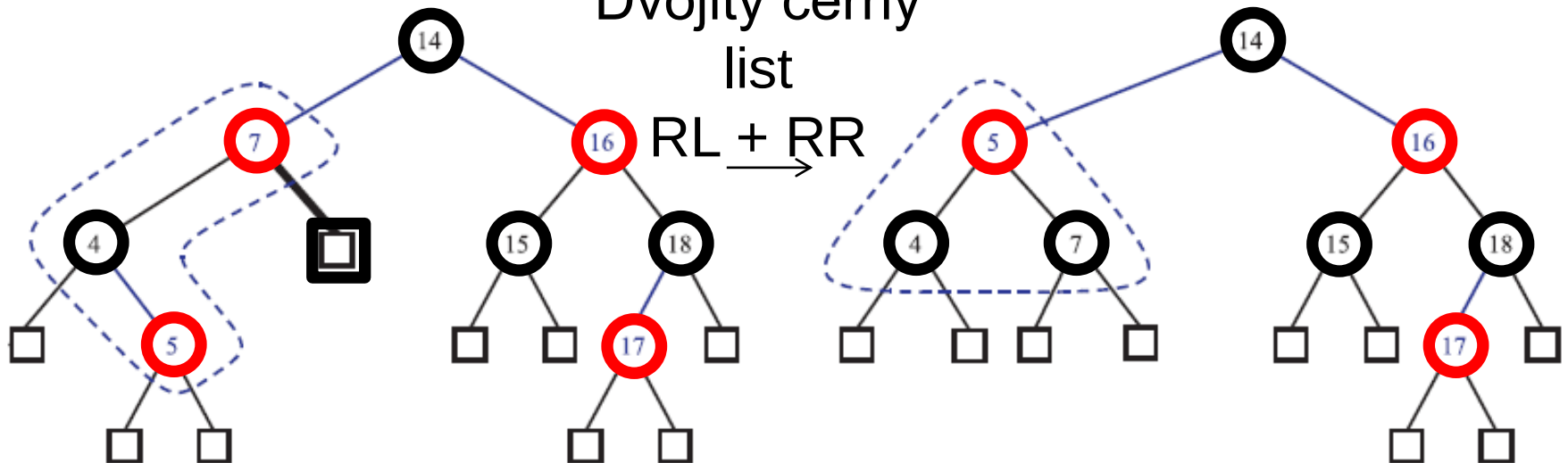
RB strom: vlož 14, 18 (+ 16, 17: nezobrazeno) 2/5



RB strom + 16 + 17: odstranění 3, 12, 17 3/5



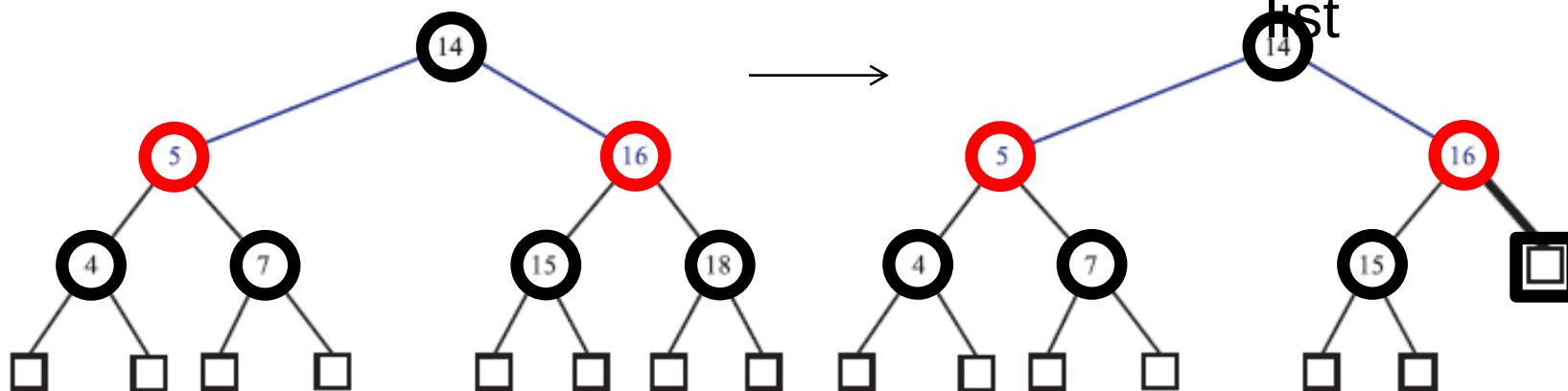
Dvojitý černý
list



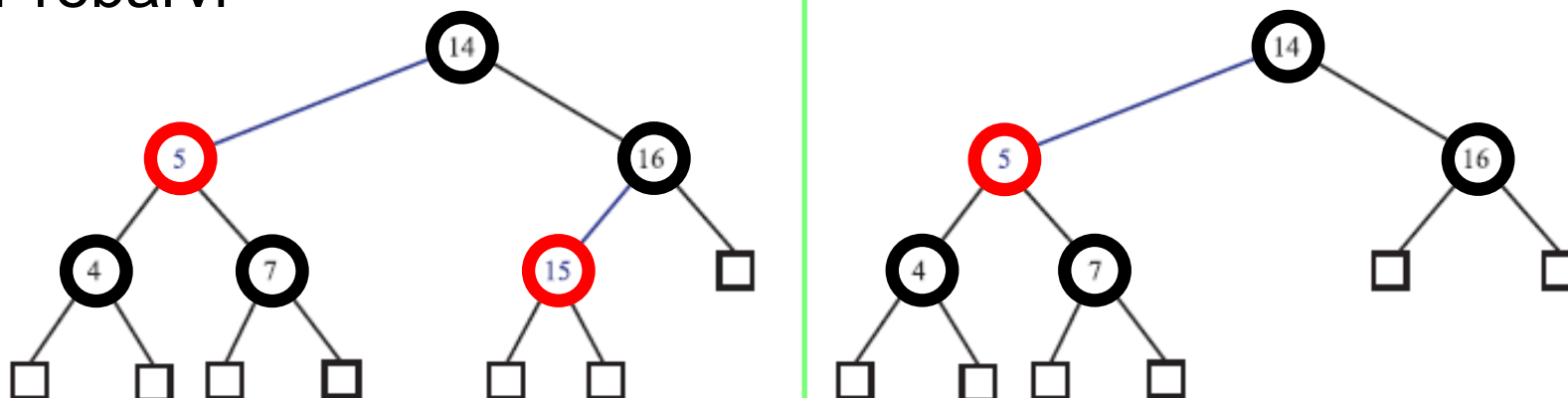
RB strom: odstranění 18, 15

4/5

Dvojitý černý list



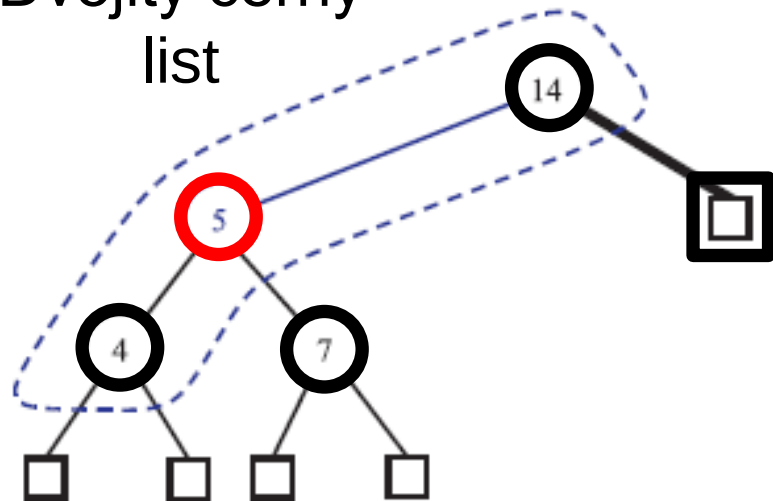
Přebarvi



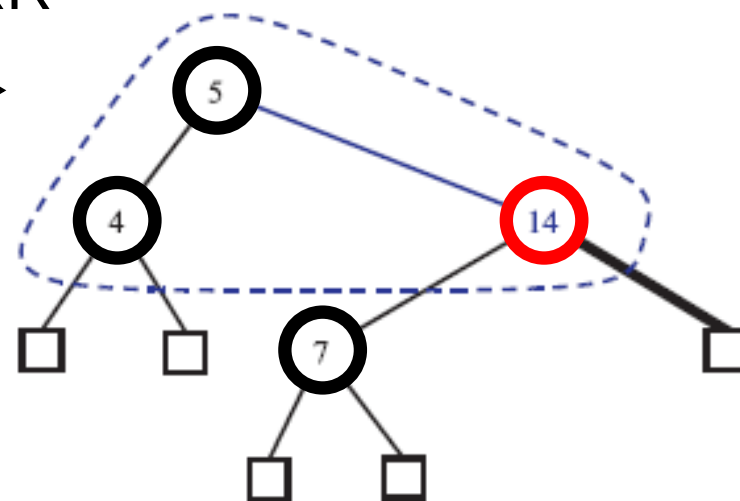
RB strom: odstranění 16

5/5

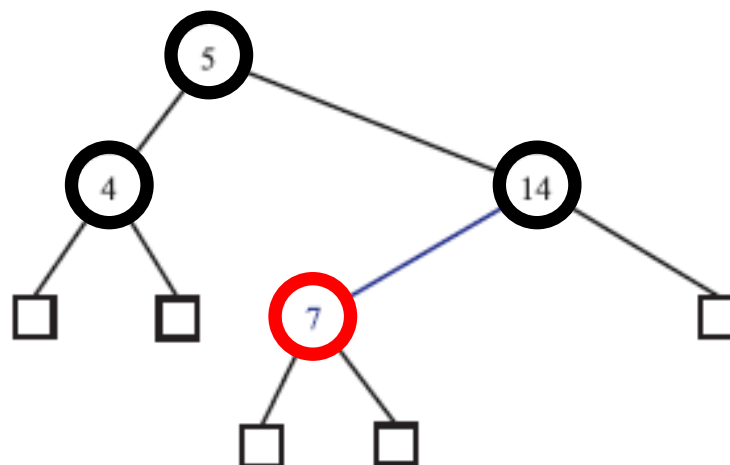
Dvojitý černý
list



RR



Přebarvi



Váhové vyvažování BVS

- Váhové vyvážení**

Váha uzlu

$$\rho(u) = 1/2$$

když u je listem,

$$\rho(u) = (|U_{UL}| + 1) / (|U_U| + 1) \quad \text{když } u \text{ je kořenem podstromu}$$

Počet uzlů v levém
podstromu

Počet uzlů ve stromu

- Strom s ohraničeným vyvážením α , $0 \leq \alpha \leq 0.5$**

pro každý jeho uzel u platí $\alpha \leq \rho(u) \leq 1 - \alpha$

- Výška stromu s ohraničeným vyvážením α**

$$h(S) \leq 1 + \log_2(n+1) - 1 / \log_2(1/(1 - \alpha))$$

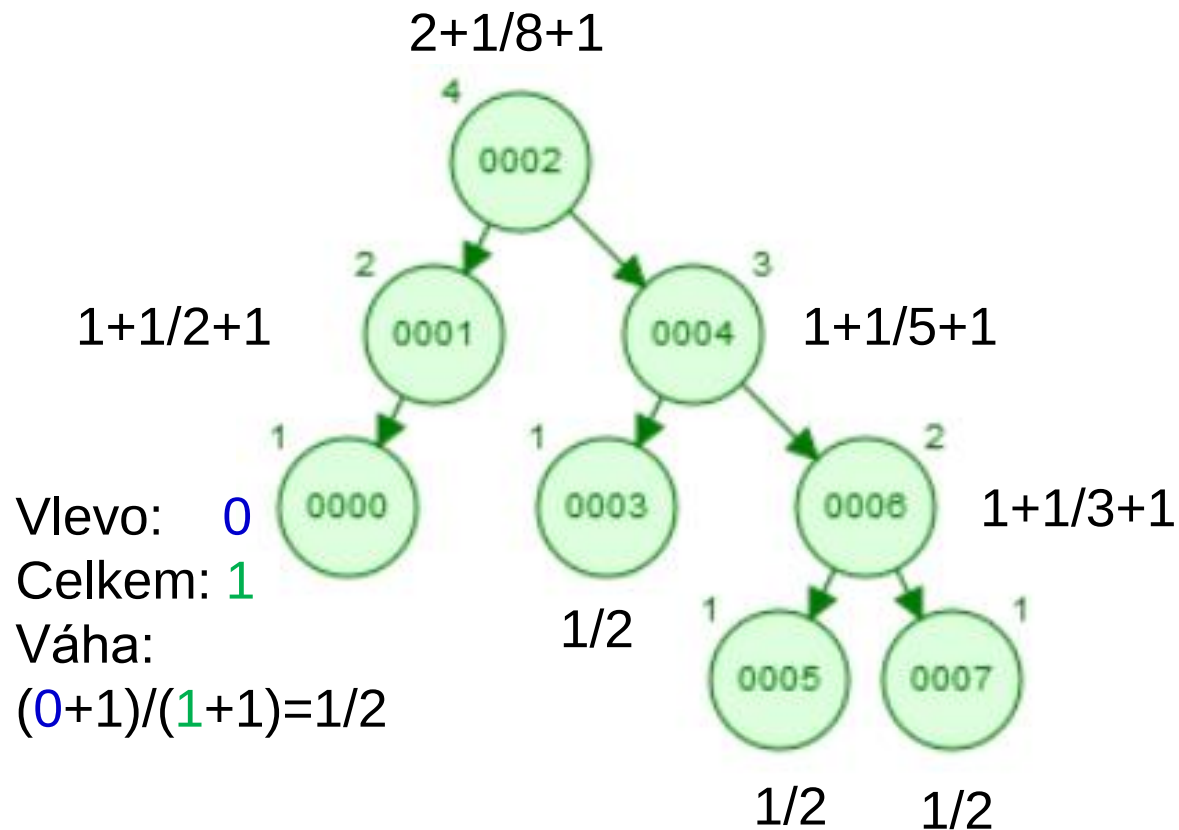
$$\text{Pro } \alpha = 1 - \sqrt{2}/2 = 0.2928$$

$$h(S) \leq 1 + (\log_2(n+1) - 1) / \log_2 2 = \log_2(n+1)$$

$\Rightarrow$ složitost hledání bude v nejhorším případě přibližně dvakrát větší než u ideálně větveného stromu

- Vyvažování stromu: pravá a levá rotace**

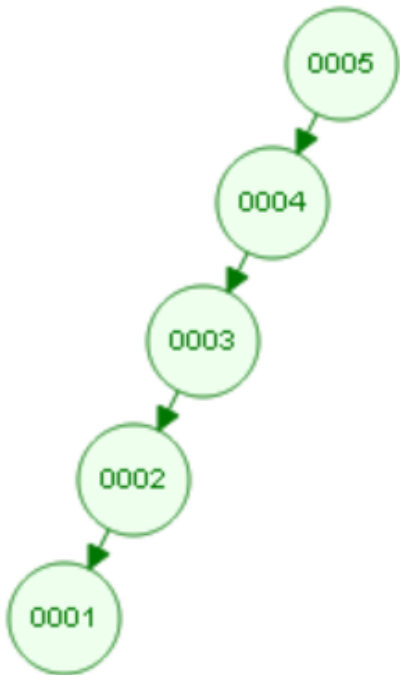
Váhově vyvážený strom, příklad



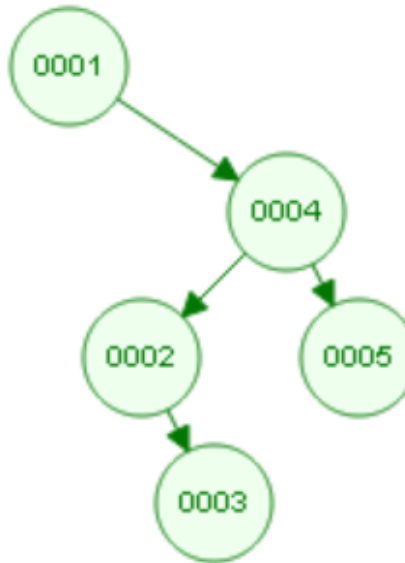
Splay strom

- Splay funkce typu **zig** a **zag** rotace vždy přesouvají aktuální (vkládaný, hledaný nebo mazaný) prvek do kořene

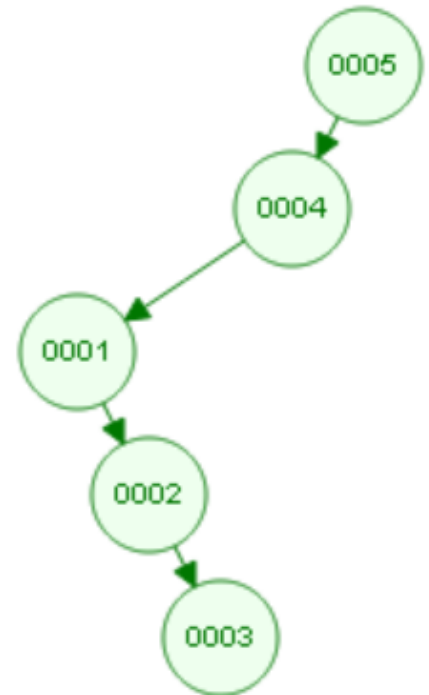
http://cs.wikipedia.org/wiki/Splay_strom



Vlož: 1, 2, 3, 4, 5



Najdi: 1



Najdi: 5

Skip list, charakteristika

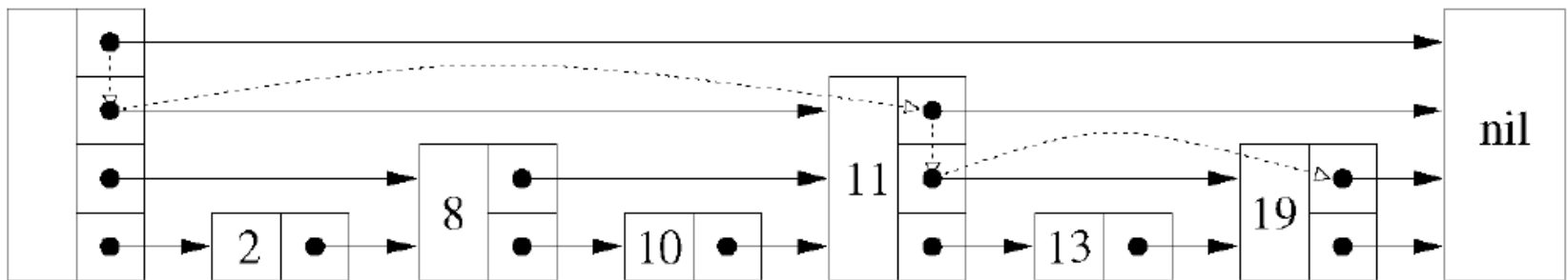
- Seznam s různě dlouhými mezerami (násobnými ukazateli), zajišťujícími operace vkládání, mazání a vyhledávání s průměrnou složitostí $O(\log(n))$

Asymptotická je vždy lineární

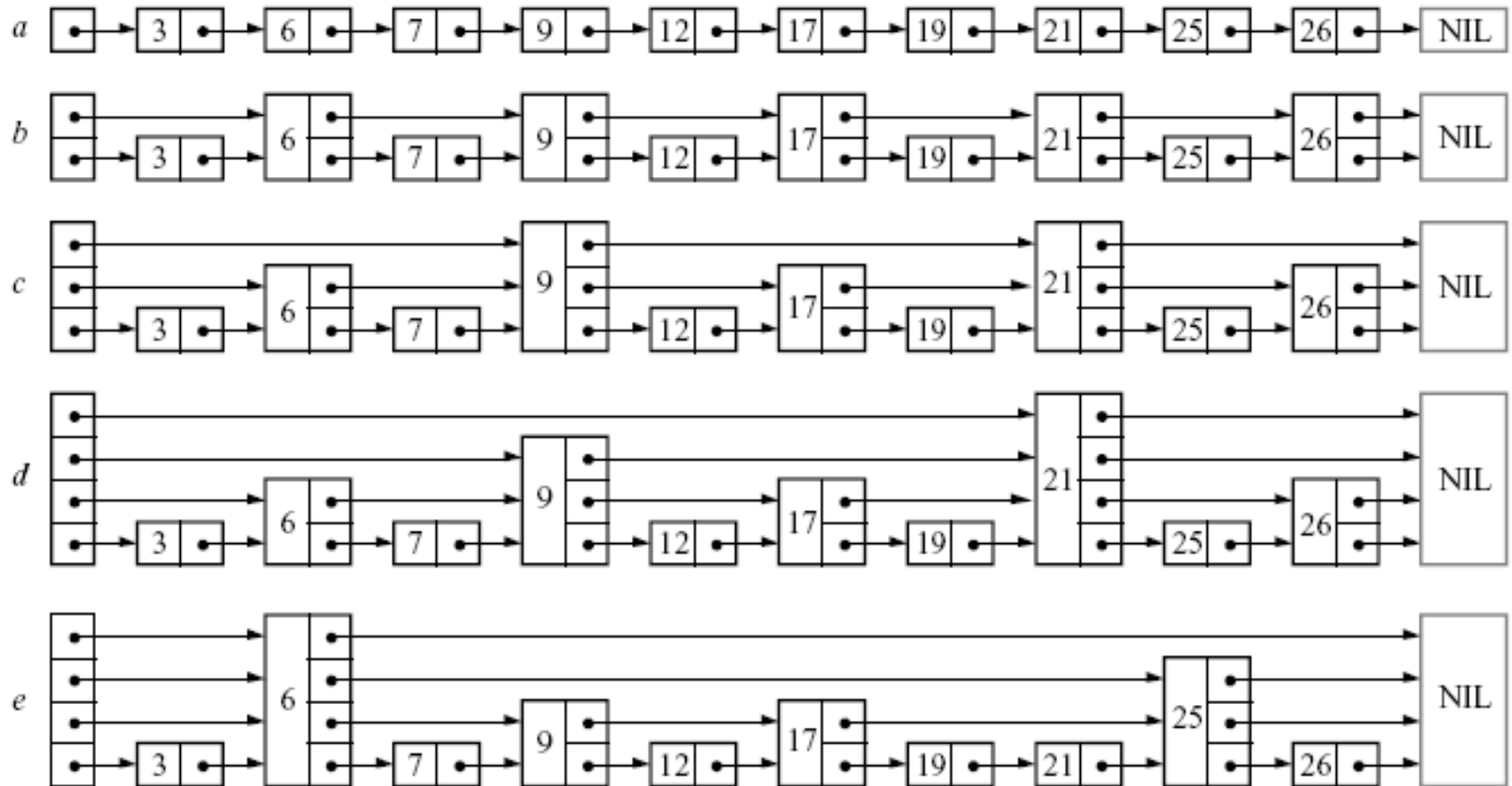
- Seznamová alternativa ke stromům
- Konstruuje se s využitím náhodných funkcí

http://en.wikipedia.org/wiki/Skip_list

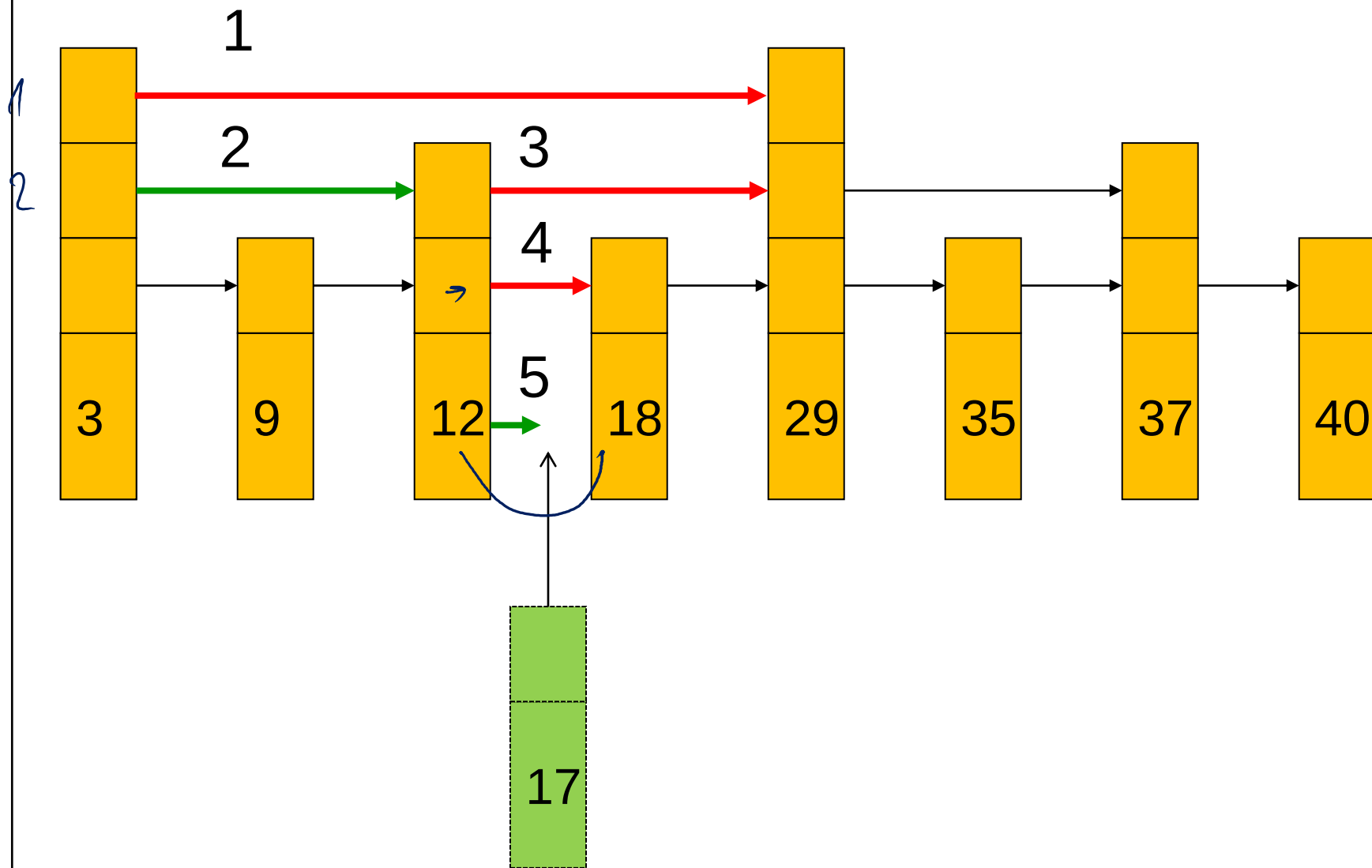
Příklad: hledání uzlu s klíčem 19 na 3 srovnání



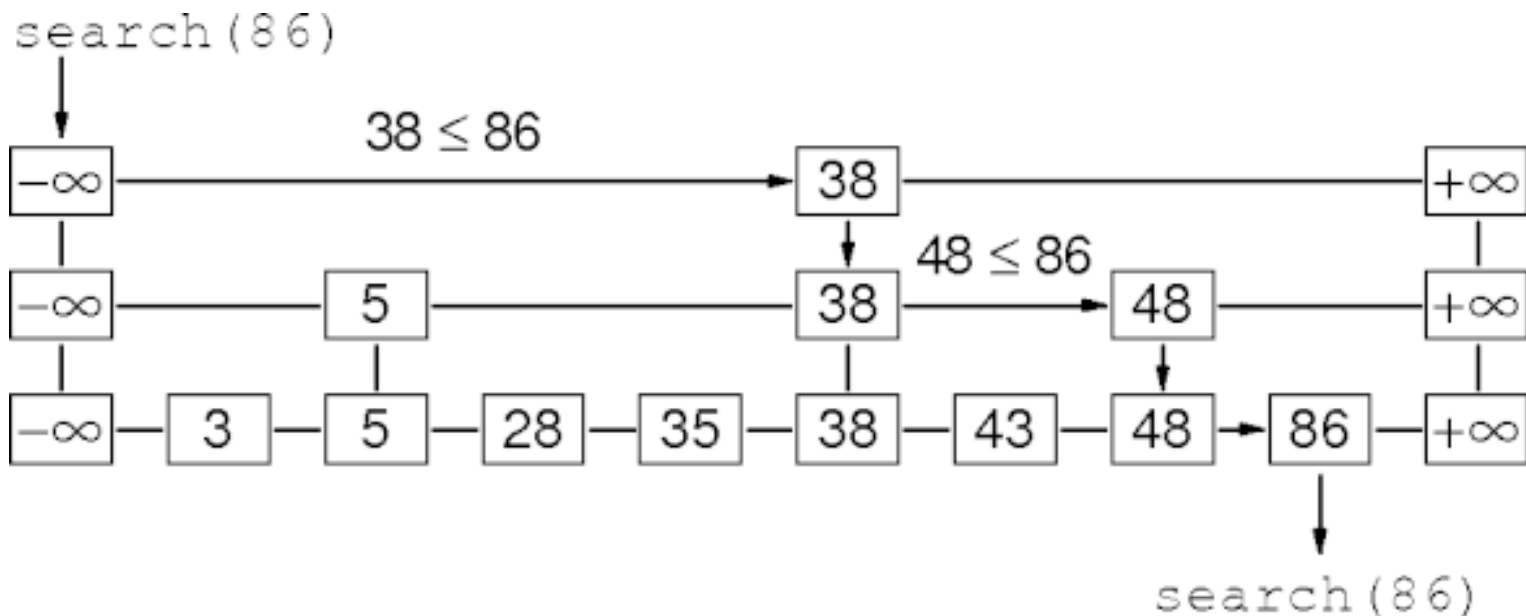
Skiplist, geneze



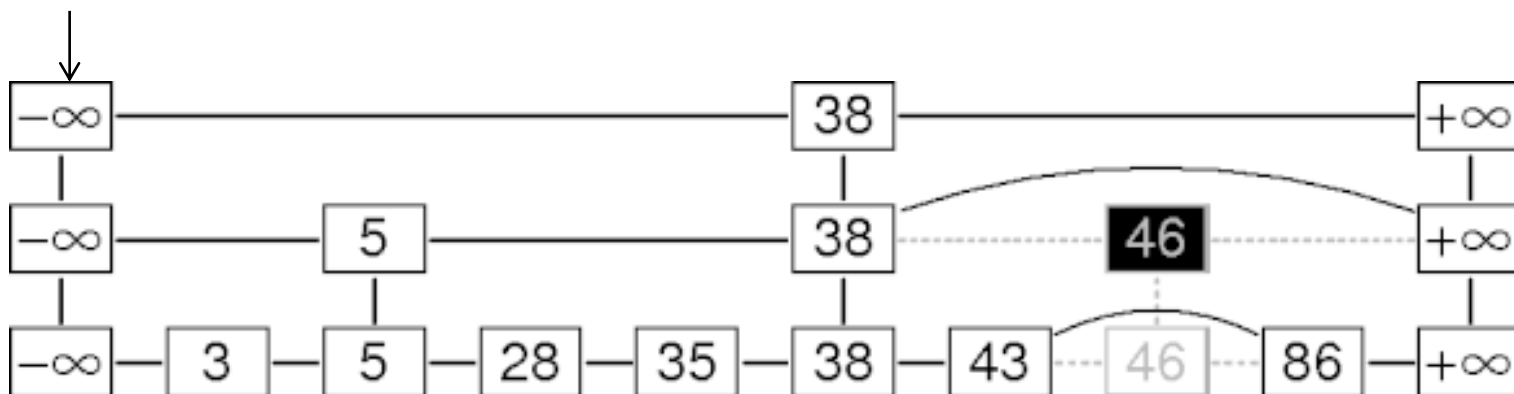
Skiplist, vkládání 17



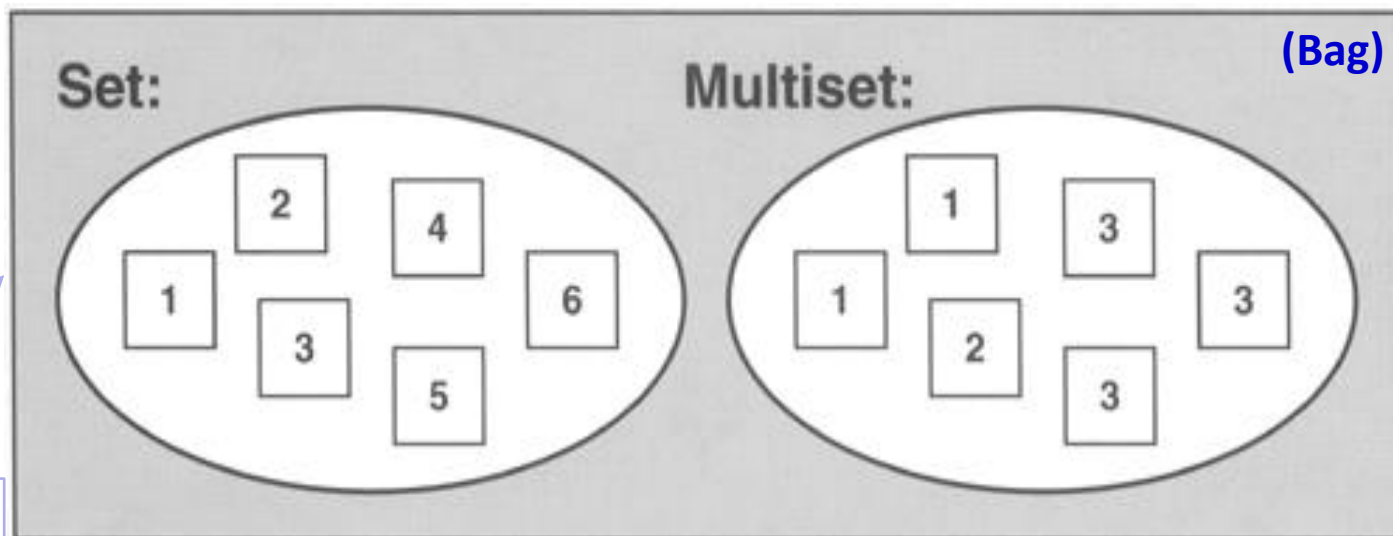
Skip list, hledání 86 a mazání 46



del(46)

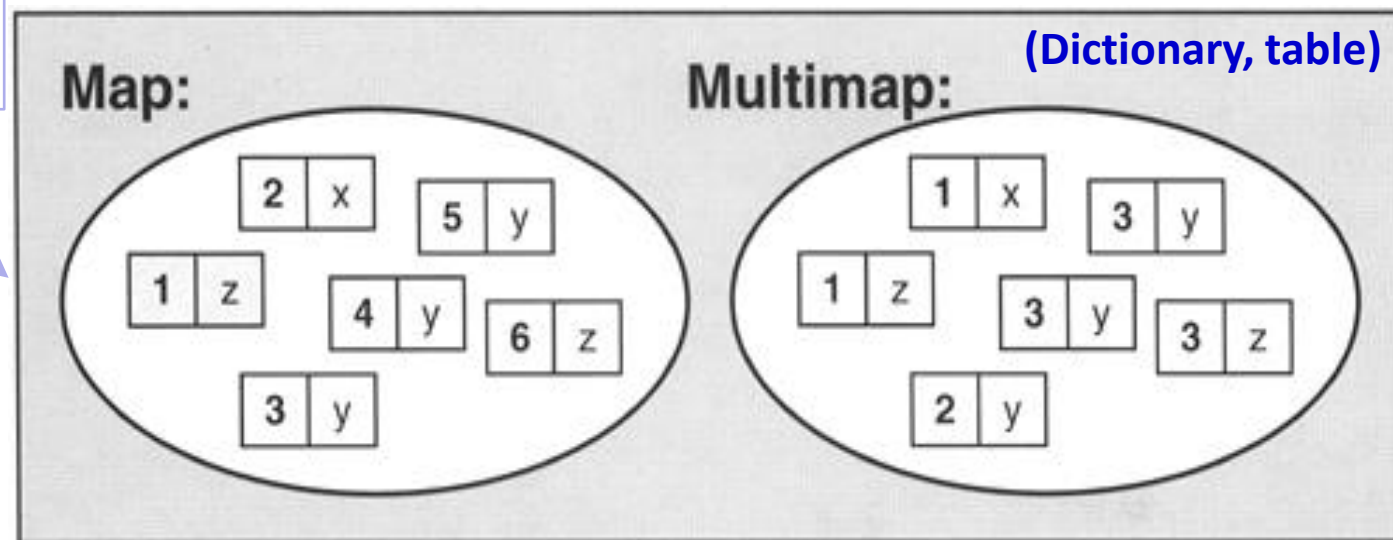


Asociativní kontejnery

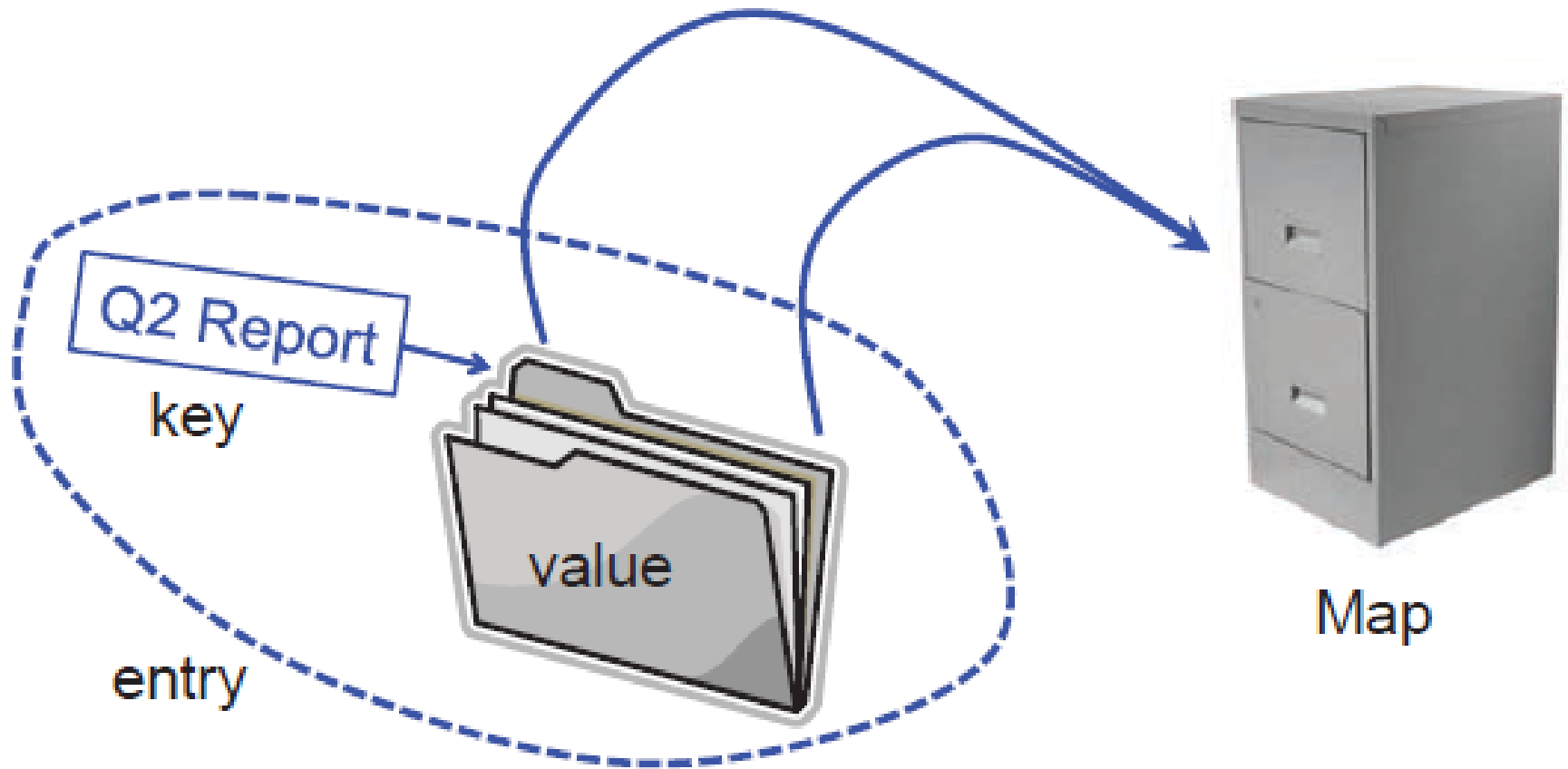


ordered_*

unordered_*



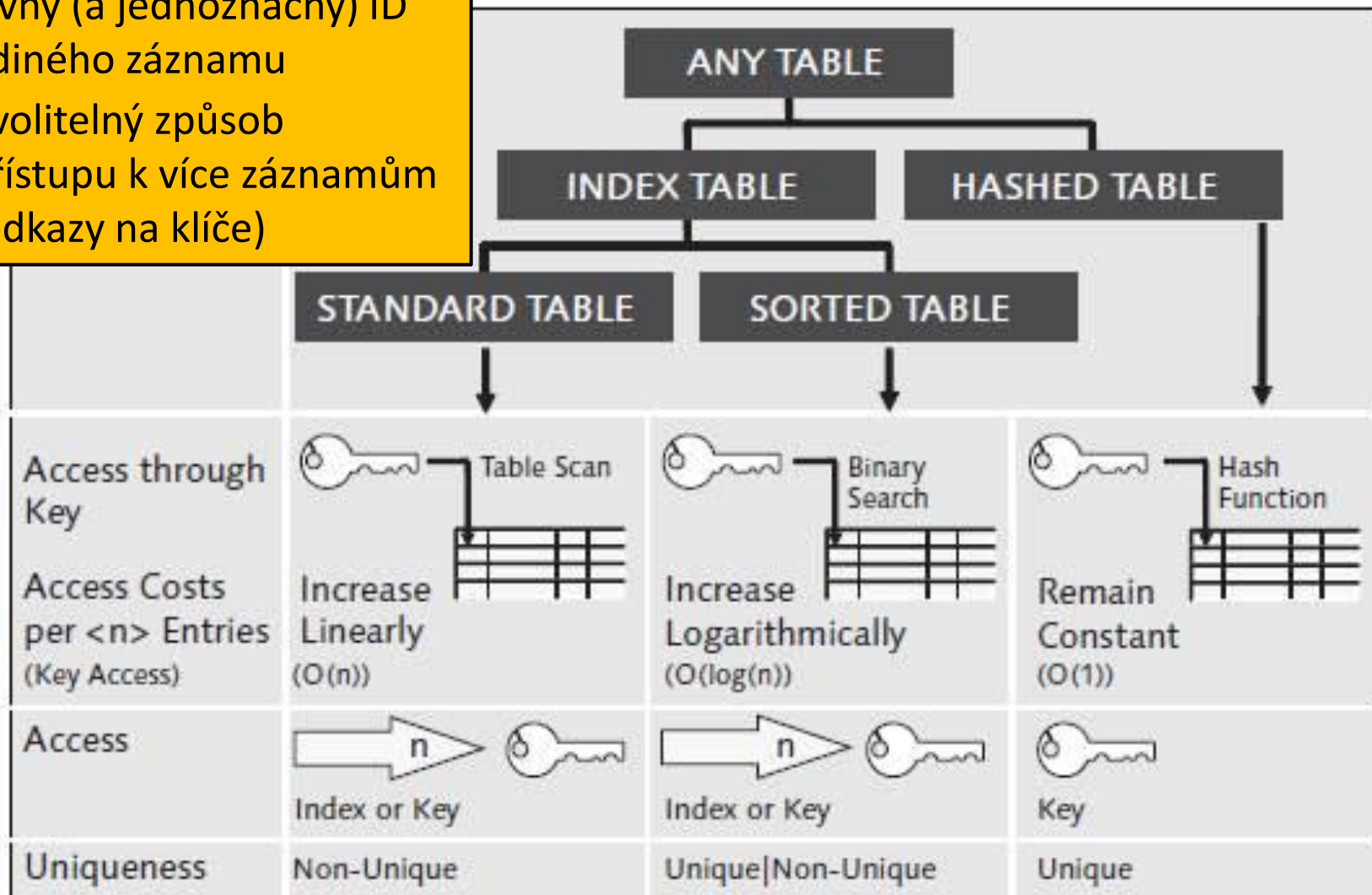
Princip mapy (tabulky)



Základní principy vyhledávání

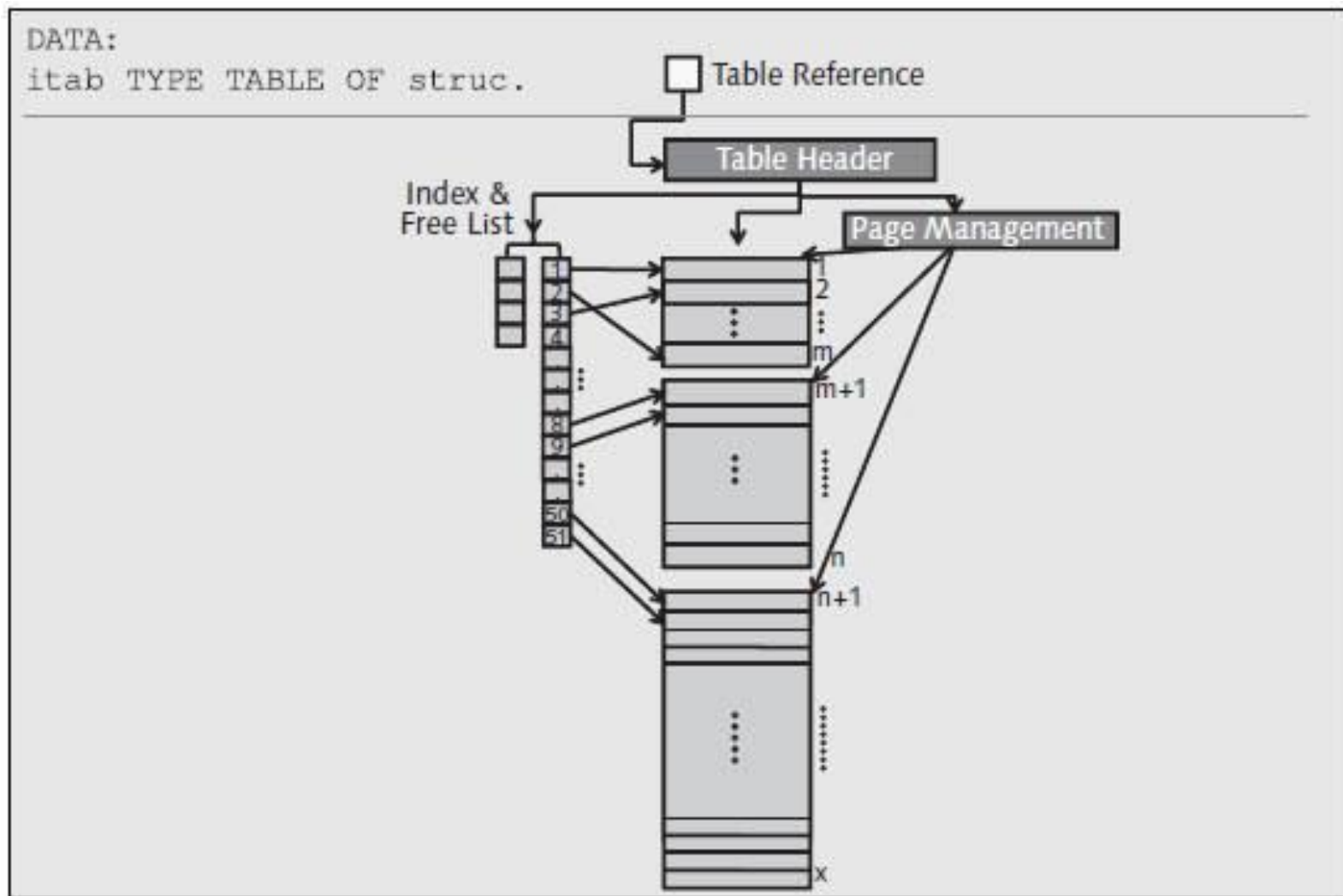
Klíč = pevný (a jednoznačný) ID
jediného záznamu

Index = volitelný způsob
přístupu k více záznamům
(odkazy na klíče)



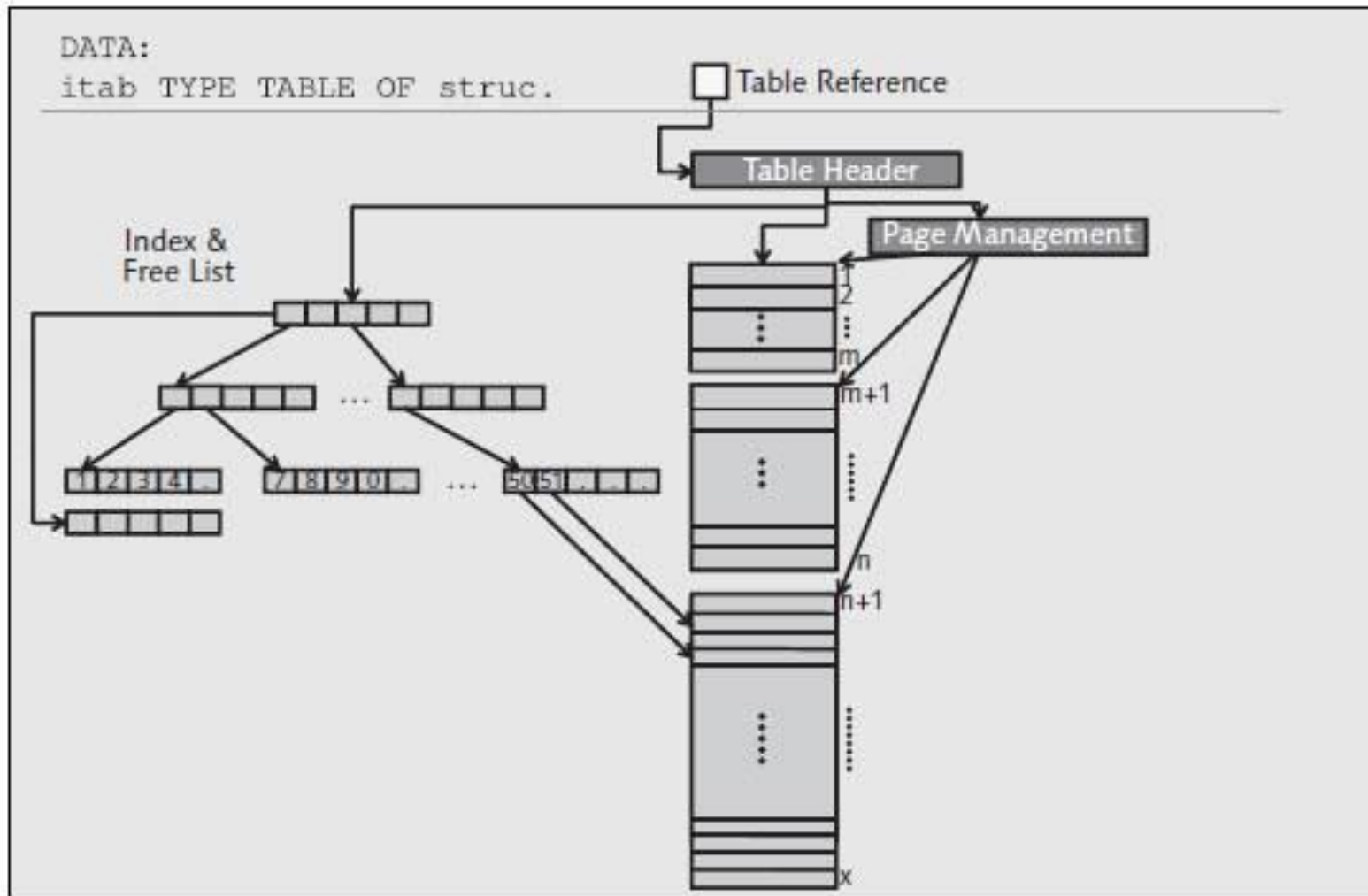
Zdroj: <http://searchsap.techtarget.com/tutorial/Introduction-to-SAP-ABAP-internal-tables>

Sekvenční (lineární) index



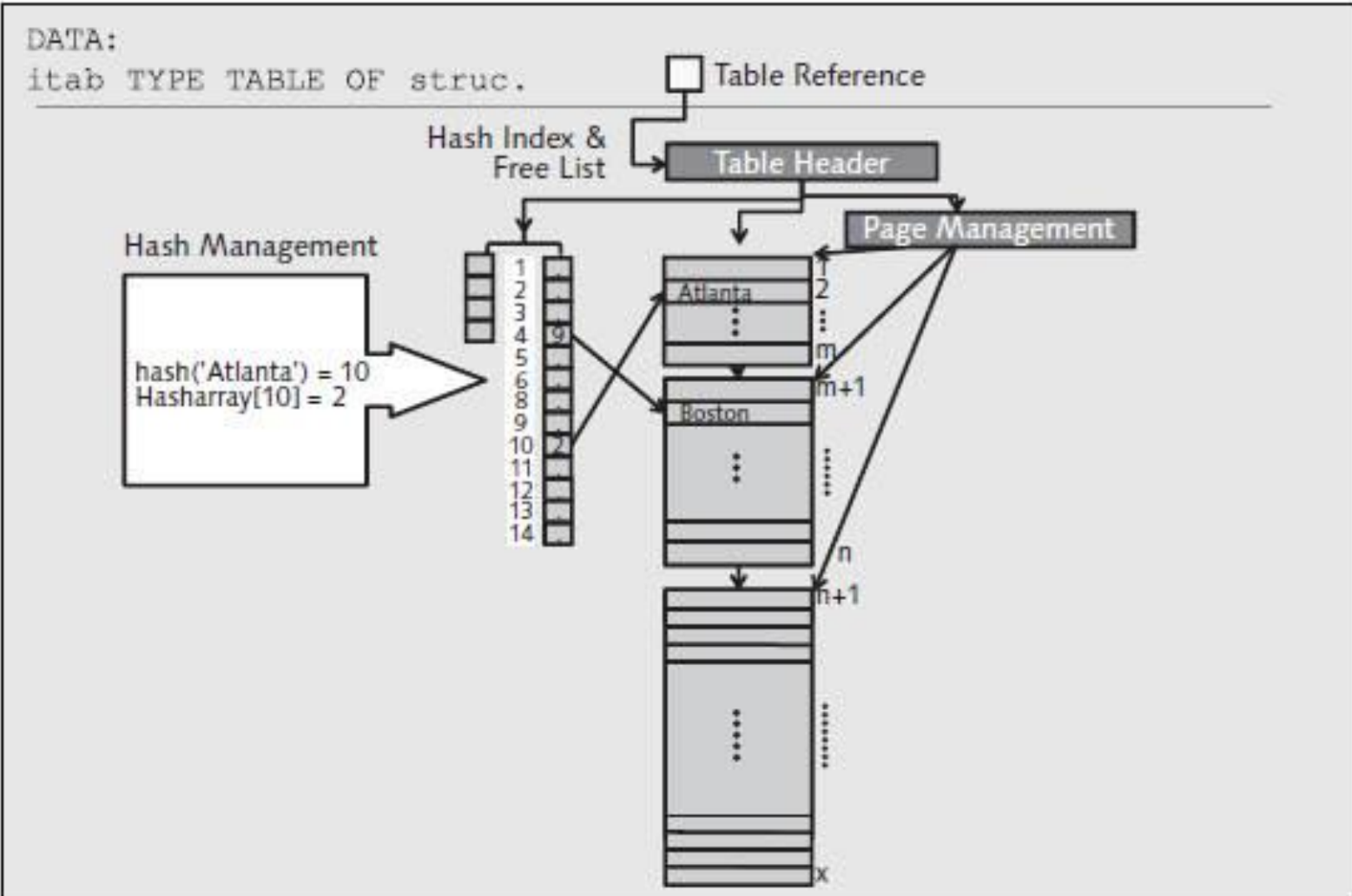
Zdroj: <http://searchsap.techtarget.com/tutorial/Introduction-to-SAP-ABAP-internal-tables>

Stromový index na seřazených klíích



Zdroj: <http://searchsap.techtarget.com/tutorial/Introduction-to-SAP-ABAP-internal-tables>

Adresní přístup klíčem – hašovací tabulka



Zdroj: <http://searchsap.techtarget.com/tutorial/Introduction-to-SAP-ABAP-internal-tables>

Vyhledávání

- Členění podle způsobu vyhledávání

Adresní vyhledávání - z klíče se vypočte adresa umístění prvku využívá se jednoznačného vztahu mezi hodnotou klíče prvku a umístěním prvku ve struktuře reprezentující vyhledávací prostor

Asociativní vyhledávání - při hledání prvku se využívají relace mezi prvky struktury reprezentující S.
(sekvenční hledání, vyhledávání ve stromech)

- Typy klíčů: `int`, `real`, `char`
- Kritéria kvality implementace
 - $P(n)$ paměťová složitost struktury reprezentující vyhledávací prostor S,
 - $Q(n)$ časová složitost operace vyhledávání,
 - $I(n)$ časová složitost operace přidání prvku,
 - $D(n)$ časová složitost operace zrušení prvku

Adresní vyhledávání

- **Přímý přístup**

výpočet adresy umístění hledaného prvku z identifikačního klíče.

Předpoklad:

ve vyhledávaném prostoru S využíváme všechny možné klíče universa U , tj. $|S| = |U|$.

Implementace S :

souvislý interval adres $A = [a_{\min} .. a_{\max}]$

souvislý interval hodnoty klíčů $[k_{\min} .. k_{\max}]$.

a: adresa prvku $p \in S$ s identifikačním klíčem k .

$f(k) = a$ - funkce která jednoznačně přiřadí každému klíči

$k \in [k_{\min} .. k_{\max}]$ adresu $a \in [a_{\min} .. a_{\max}]$

Příklad: $f(k) = (k - k_{\min}) \cdot d + a_{\min}$,

d - délka zobrazení prvku vyhledávacího prostoru

Operační složitost: $Q(n) = I(n) = D(n) = O(1)$

Adresní vyhledávání

- Připomeňme si nejefektivnější metodu implementace množiny a tabulky:
 - jsou-li hodnotami klíčů celá čísla z intervalu 0 až *max*, lze pro implementaci použít pole, jehož indexy jsou klíče a hodnotou prvku je v případě množiny *true* nebo *false*, v případě tabulky to jsou hodnoty odpovídající klíčům
 - vyhledávání se v tomto případě nazývá *adresním vyhledáváním s přímým přístupem* (z klíče lze určit adresu umístění prvku v datové struktuře) a jeho složitost je $O(1)$

Rozptylování

- Jestliže nelze stanovit vzájemně jednoznačné zobrazení mezi klíči a adresami umístění v datové struktuře, lze použít techniku **rozptylování** (hashing)
- Princip rozptylování (patří mezi adresní vyhledávání):
 - pro klíč se vypočte adresa, zobrazení mezi klíči a adresami však není vzájemně jednoznačné (několika různým klíčům odpovídá stejná adresa)
 - klíče, kterým odpovídá stejná adresa, se nazývají synonyma, a je pro ně třeba stanovit náhradní způsob umístění
- Funkce pro výpočet (základní) adresy umístění klíče se nazývá **rozptylovací funkce** (hash function)

Rozptylování

- **Vyhledávání s rozptylováním**

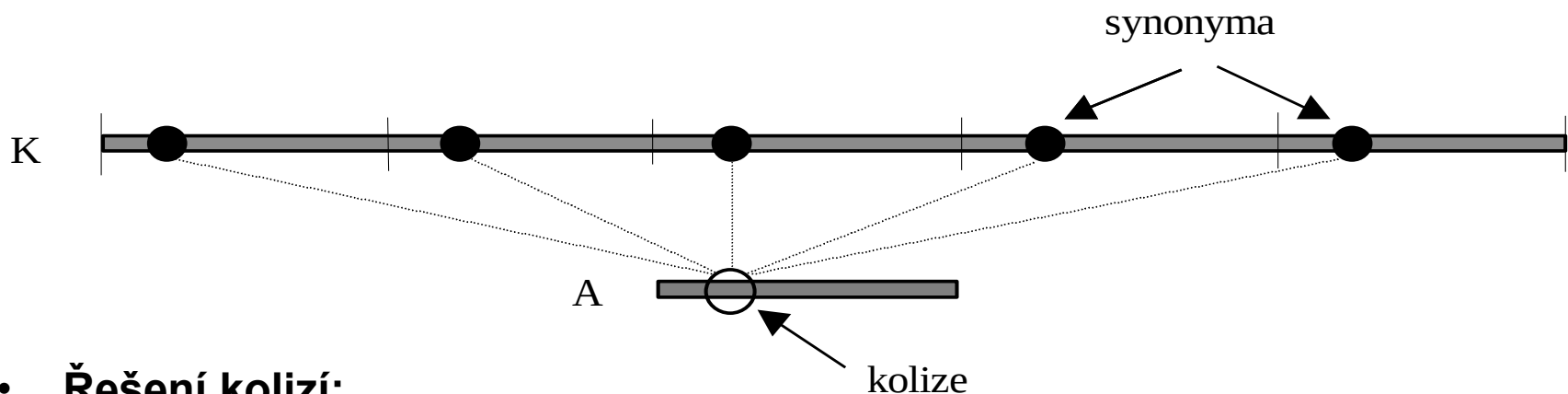
K - množina identifikačních klíčů,

$A = [a_{\min} .. a_{\max}]$ - souvislý interval adresních klíčů, $|K| \gg |A|$.

- **rozptylovací funkce**

$h(k) = a, k \in K, a \in A$

- **synonyma:** $k_1 \neq k_2$, pro něž platí $h(k_1) = h(k_2) \Rightarrow$ **kolize**



- **Řešení kolizí:**

zřetězené rozptylování - zřetězení synonym v dynamické paměti

otevřené rozptylování - sekvenční ukládání do pole

Rozptylovací funkce

- Předpokládejme, že klíče chceme transformovat na indexy pole z intervalu 0 až $m-1$ (pole obsahuje m prvků)
- Příklad 1: klíče jsou nezáporná celá čísla

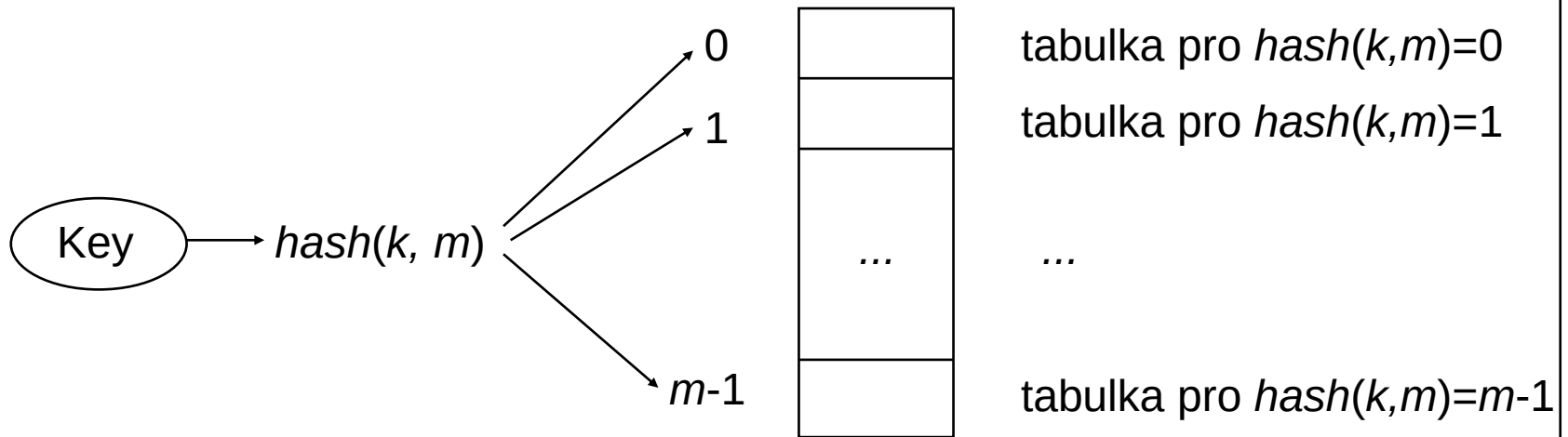
```
int hash(int k, int m) {  
    return k % m;  
}
```

- Příklad 2: klíče jsou řetězce

```
int hash(const char* s, int m) {  
    int h = 0, len = strlen(s);  
    for (int i=0; i<len; i++)  
        h += s[i];  
    return h % m;  
}
```

Interpolační rozptylování

- Pole tvořené m prvky obsahuje m dílčích tabulek, v tabulce s indexem i jsou synonyma, pro které rozptylovací funkce dá výsledek i



- Dílčí tabulku lze realizovat různým způsobem, např.
 - spojovým seznamem
 - uspořádaným spojovým seznamem
 - rozšiřitelným polem
 - atd.

Interpolační rozptylování

- Realizace dílčích tabulek pomocí spojového seznamu
- **Šablona pro spojový seznam:**

```
// lnk1st.h
#include <iostream>
using namespace std;
template <class T>
class LinkedList {                                // prvek seznamu
    struct Elem {
        T key;
        Elem *next;
        Elem(T v, Elem *n) {
            key = v; next = n;
        }
    };
    Elem *head;                                  // začátek seznamu
};
```

Interpolační rozptylování

```
public:
    LinkedList() {head = NULL;}    // prázdný seznam
    bool isEmpty() const { // test na prázdný seznam
        return head == NULL;
    }
    void add(T x) { // přidání prvku na začátek seznamu
        if (!incl(x))
            head = new Elem(x, head);
    }
    void del(T x) {
        Elem *p, *pp;
        p = head;
        pp = NULL;
        while (p && p->key!=x) { // hledání prvku
            pp = p;
            p = p->next;
        }
        if (p==NULL) return; // prvek nenalezen
        if (pp==NULL)
            head = p->next;    // ruší se první prvek seznamu
        else
            pp->next = p->next;
        delete p;
    }
}
```

Interpolační rozptylování

- Pokračování šablony pro spojový seznam

```
bool incl(T x) {                                // test existence prvku x
    Elem *p = head;
    while (p)
        if (p->key==x)
            return true;
        else p = p->next;
    return false;
}

friend ostream& operator<<(ostream& s,
                           const LinkList& l) {

    Elem *p = l.head;
    while (p) {
        s << p->key << ' ';
        p = p->next;
    }
    return s;
}
};
```

Interpolační rozptylování

- Deklarace třídy **HashSet** -množiny celočíselných klíčů

```
// hashset.h

#include "hash.h"
#include "lnklst.h"
#include <iostream>
using namespace std;

typedef int Key;           //celočíselný klíč

class HashSet {
    int m;
    LinkedList<Key> *keys;  //pole prázdných seznamů

public:
    HashSet(int m);        //konstruktor prázdné množiny
    ~HashSet();            //destruktor prázdné množiny
    void ins(Key);
    bool incl(Key) const;
    void del(Key);
    void print() const;
};
```

Interpolační rozptylování

- Metody třídy *HashSet*:

```
HashSet::HashSet(int m) {  
    this->m = m;  
    keys = new LinkedList<Key>[m]; //pole prázdných seznamů  
}  
// destruktor množiny  
HashSet::~~HashSet() {  
    delete [] keys;  
}  
// vloží prvek do i-tého seznamu synonym  
void HashSet::ins(Key k) {  
    int i = hash(k, m);  
    keys[i].add(k);  
}  
// existuje prvek v seznamu synonym?  
bool HashSet::incl(Key k) const {  
    int i = hash(k, m);  
    return keys[i].incl(k);  
}
```

Interpolační rozptylování

- Metody třídy *HashSet* (pokračování):

```
// ruší prvek v seznamu synonym
void HashSet::del(Key k) {
    int i = hash(k, m);
    keys[i].del(k);
}

// tisk množiny
void HashSet::print() const {
    for (int i=0; i<m; i++)
        if (!keys[i].isEmpty()) {
            cout << setw(3) << i << ": ";
            cout << keys[i] << endl;
        }
    cout << endl;
}
```

Adresní vyhledávání - **otevřené rozptylování**

- **Otevřené rozptylování**

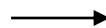
Vyhledávací prostor je reprezentován pouze polem prvků V

Zařazování prvku do pole V - prvek se umístí na první volnou pozici počínaje $h(k)$

Hledání prvku - postupně se prohlížejí pozice počínaje $h(k)$ až do nalezení hledaného prvku.

Příklad: $S = \{ 2, 9, 16, 3, 6, 1, 8 \}$

$$h(k) = k \bmod 7$$



0	08
1	01
2	02
3	09
4	16
5	03
6	06

Otevřené rozptylování

- Příklad: S je množina celočíselných klíčů

```
typedef int Key;
const int none = 0xFFFFFFFF;

class HashSet {
    int m;
    int n;
    Key *keys;

public:
    HashSet(int m);
    ~HashSet();
    void ins(Key);
    bool incl(Key) const;
    void del(Key);
    void print() const;
};
```


Otevřené rozptylování

- Příklad (pokračování):

```
HashSet::HashSet(int m) {  
    this->m = m;  
    n = 0;  
    keys = new Key[m];           // pole prvků  
    for (int i=0; i<m; i++)  
        keys[i] = none;         // nulování pole  
}  
  
HashSet::~~HashSet() {  
    delete [] keys;  
}
```

Otevřené rozptylování

- Příklad (pokračování):

```
void HashSet::ins(Key k) {
    int i = hash(k, m);
    // najde první volný počínaje i-tým
    while (keys[i] != none && keys[i] != k)
        i = (i+1) % m;
    if (keys[i] == k) return;
    n++; // zvýší počet prvků a vloží prvek
    keys[i] = k;
}

bool HashSet::incl(Key k) const {
    int i = hash(k, m);
    // počínaje i-tým hledá do nalezení prvku
    // nebo do prvního prázdného prvku
    while (keys[i] != none && keys[i] != k)
        i = (i+1) % m;
    if (keys[i] == k) return true; // úspěšné hledání
    return false; // neúspěšné hledání
}
```

Otevřené rozptylování

- Příklad (pokračování):

```
void HashSet::del(Key k) {
    int i = hash(k, m);
    // hledá se prvek
    while (keys[i]!=none && keys[i]!=k)
        i = (i+1) % m;
    if (keys[i]==none) return; //prvek není

    keys[i] = none; // nalezený prvek se zruší
    n--;

    // následná synonyma se znovu vloží(proč_?)
    i = (i+1) % m;
    while (keys[i]!=none) {
        n--; k = keys[i];
        keys[i] = none;
        ins(k);
        i = (i+1) % m;
    }
}
```

Otevřené rozptylování

- Příklad (pokračování):

```
void HashSet::print() const {  
    cout << "n=" << n << " faktor zaplneni " <<  
    setprecision(2)  
        << (float)n/m << endl;  
    for (int i=0; i<m; i++)  
        if (keys[i]!=none)  
            cout << keys[i] << ' '  
    cout << endl;  
}
```

Poznámka:

V uvedeném příkladu se uvolnění paměti po zrušení prvku řeší novým vložením následných synonym.

Jiné řešení: každý prvek pole má příznak prázdný/plný, který se využije při hledání a vkládání prvků

⇒ nižší operační složitost operace del
vyšší průměrná složitost operace incl

Otevřené rozptylování

- **Dvojité rozptylování**

V předchozím příkladě se synonyma ukládala do dalších prvků pole s krokem $k=1$

⇒ možnost velkých shluků synonym

⇒ zhoršení operační složitosti vyhledávání

Řešení:

1. Volíme krok $k > 1$

(Nejlépe k = prvočíslo nesoudělné s m)

2. Dvojité rozptylování

$$h(k,i) = [h_1(k) + i \cdot h_2(k)] \bmod m$$

Příklad:

$$h(k,i) = [k \bmod m + 3 \cdot i] \bmod m,$$

$i = 0, 1, 2, \dots, m - \text{délka pole prvků}$

Otevřené rozptylování

- **Operační složitost**

Průměrná složitost závisí na **poměrném zaplnění** $\beta = n / m$

n - počet uložených prvků

m - délka reprezentujícího pole V

Složitost vkládání: $I(n,m) = (m + 1)/(m - n + 1) \approx 1/(1 - \beta)$

Složitost hledání : $Q(n,m) = 1/\beta \cdot \ln(1/(1 - \beta))$

β	0.5	0.7	0.8	0.9	0.95	0.99
$I(n,m) = 1/(1-\beta)$	2.0	3.3	5.0	10.0	20	100
$Q(n,m) = 1/\beta \cdot \ln(1/(1 - \beta))$	1.38	1.70	2.01	2.55	3.15	4.65

Otevřené rozptylování

- Úprava tabulky při vysokém β
 1. Zvýšit rozsah m
 2. Upravit rozptylovací funkci $h(k)$ pro nový rozsah
 3. Všechny dříve uložené prvky uložit znovu do nového pole V podle upravené rozptylovací funkce.

Adresní vyhledávání

- **Požadavky na rozptylovací funkci $h(k)$**

1. $h(k) \in [0 .. m]$ pro všechna $k \in K$
2. funkce $h(k)$ vytváří minimální počet kolizí (generuje minimum synonym).
3. rozptýlení na intervalu $[0 .. m - 1]$ je rovnoměrné.

Adresní vyhledávání

- **Realizace funkce** $h_1(k)$:

1. **Část klíče** k

Příklad:

k - celé desítkové číslo v rozsahu $0 \leq k \leq 10^6$,
 $m = 10^3$. Za i můžeme zvolit např. 2. až 4. řád k .

- 2 . **Část výsledku operace** nad k

Příklad: část $\sqrt{k}$

3. $k \bmod m$ (zbytek po celočíselném dělení)

4. $h(k) = (k \bmod N) + q$

N - prvočíslo nejbližší nižší rozsahu m

q – prvočíslo nesoudělné s m

- 5 . **Váhový součet** částí k^j , $j = 1, 2, \dots, r$

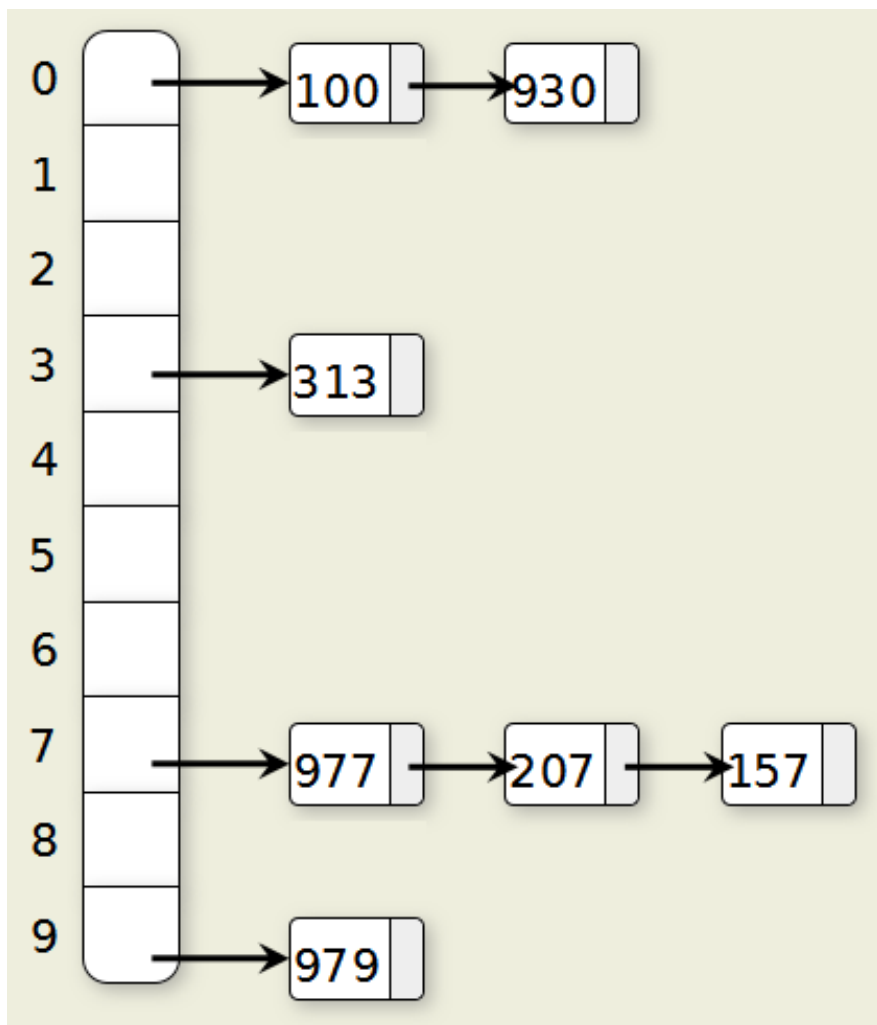
$$h(k) = (\sum_j a_j \cdot k^j + q) \bmod m$$

a_j - **vhodně volené váhy**

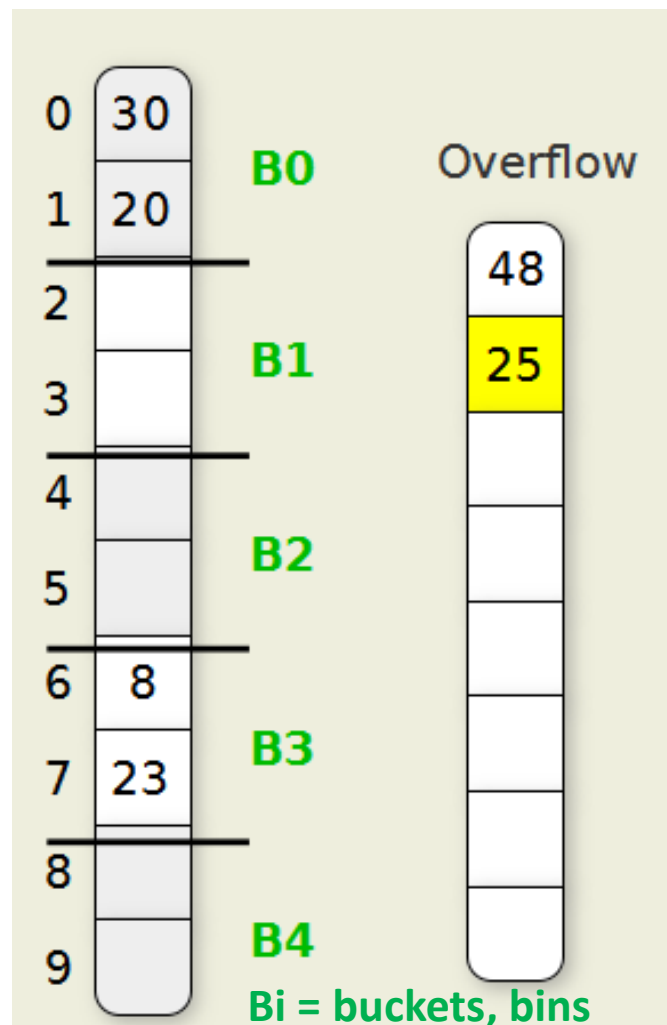
Hašovací tabulky (t. s rozptýlenými parametry)

- Rozptylovací (Hash) funkce – příhodná transformace klíče na index v H
- Kolize – některé klíče se namapují na stejný index v H.
Řešíme je:
 - **Řetězením** (interpolační, [separate chaining](#)) – kolizní hodnoty uložíme mimo tabulku, např. do spojového seznamu nebo overflow buffer_u
 - **Otevřeným adresováním** ([open addressing](#), [closed hashing](#)) – kolize řešíme zavedením metod, které je v tabulce jednoznačně realokují na jiné místo
 - Probe sequences

Řetězení



% 10



% 5

Konflikty v otevřeném adresování a jejich řešení

Lineární posun adresy

- Vstup: 9877, 9050, 2037, 1059, 7200

$$h_{LP}(k, i) = (h(k) + i) \bmod m \quad \text{kde}$$

$$h(k) = k \bmod m$$

$i = 0, 1, 2, \dots, m-1$ je offset /

číslo kolize/pořadí první volné pozice

- Co vstup 3348?

– **Problém: vznik shluků**

9050	7200						9877	2037	1059
0	1	2	3	4	5	6	7	8	9

Dosáhnout vyváženějšího zaplnění pole

- 1) Zvýšit lineární inkrement (+2, +3 apod.)
- 2) Zvolit pseudonáhodný posun: konstantní nebo proměnný
- 3) Kvadratický posun

$$h_{KP}(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

- Příklad: vstupem je 5, 5, 5 se „zkušební adresací“ ([probing](#)) i^2+1 :

5+1, 5+4, 5+9

- Dosáhneme tak rovnoměrnějšího zaplnění tabulky

4) Zvolit vícenásobnou hašovací funkci

Například dvojité hašování:

$$h_{DH}(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

55	24	11					66			
0	1	2	3	4	5	6	7	8	9	10

Příklad: kromě velikosti tabulky (m) využijeme i hodnotu klíče k

- $h_1 = k \bmod m$
- $h_2 = 1 + (k \bmod (m-1))$

Vstup: 55, 66, 11, 24, $m = 11$

- $h_1(55) = 0, i=0$
- $h_1(66) = 0, h_2(66) = 1+(66\%10) = 7, i=1$
- $h_1(11) = 0, h_2(11) = 1+(11\%10) = 2, i=1$
- $h_1(24) = 2, h_2(24) = 1+(24\%10) = 5, i=2$

bez kolize

1 kolize (55)

1 kolize (55)

2 kolize (22,66)

Ukázka práce s tabulkou, M = 10

- Vlož: 5, 26, 75 →

					5	26	75		
0	1	2	3	4	5	6	7	8	9

- Odstraň: 26 →

					5	[del]	75		
0	1	2	3	4	5	6	7	8	9

- Odstraň: 75 →

					5	[del]	[Del]		
0	1	2	3	4	5	6	7	8	9

- Vlož: 97 →

					5	[del]	97		
0	1	2	3	4	5	6	7	8	9

- Hledej 96 →

					5	[del]	97		
0	1	2	3	4	5	6	7	8	9

Hašovací tabulka, příklad 1

Do hašovací tabulky o velikosti $m=11$ a funkcemi:

$$h_1 = \text{mod } m$$

$$h_2 = 1 + (k \bmod (m-1))$$

$$h_{DH}(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

zařadíte následující klíče:

22, 1, 13, 11, 24, 33, 18, 42, 31

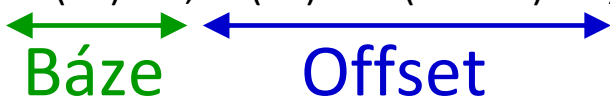
DH:

$h_1(11) = 0$, $h_2(11) = 1 + (11 \% 10) = 2$, $i=2$ 2 kolize (22,13)

$h_1(24) = 2$, $h_2(24) = 1 + (24 \% 10) = 5$, $i=1$ 1 kolize (13)

$h_1(33) = 0$, $h_2(33) = 1 + (33 \% 10) = 4$, $i=2$ 2 kolize (22,11)

$h_1(31) = 9$, $h_2(31) = 1 + (31 \% 10) = 2$, $i=4$ 4 kolize (42,22,13,11)


Báze Offset

	Chain	LP	DH
0	33->11->22	22	22
1	1	1	1
2	24->13	13	13
3		11	
4		24	11
5		33	18
6			31
7	18	18	24
8			33
9	31->42	42	42
10		31	

Hašovací tabulka, příklad 2

Do hašovací tabulky o velikosti $m=10$ a funkcí:

$$h_{QP}(k, i) = (h(k) + i^2) \bmod m$$

zařadíte následující klíče:

89, 18, 49, 58, 69, 78

0	49
1	
2	58
3	69
4	
5	
6	
7	78
8	18
9	89

Hašovací tabulka, příklad 3

(řešení: žlutá + oranžová)

Do hašovací tabulky pro 13 prvků s dvojitým hašováním, realizovaným těmito funkcemi:

$$h1(k) = k \% 13$$

$$h2(k) = 7 - k \% 7$$

zařadíte následující klíče:

18, 41, 22, 44, 59, 32, 31 a 73

$$h1(44) = 5; h2(44) = 5, i = 1; (5 + 5)\%13 = 10$$

$$h1(31) = 5; h2(31) = 4, i = 2; (5 + 2*4)\%13 = 0$$

0	31
1	
2	41
3	
4	
5	18
6	32
7	59
8	73
9	22
10	44
11	
13	